

Sistemi Operativi

Marco Tartaglione

7 ottobre 2026

Indice

1	Introduzione	9
1.1	Hardware	9
1.2	Aspetti che differenziano i sistemi operativi	9
1.3	Sviluppi storici	9
1.3.1	Sistemi batch Semplici	9
1.3.2	Sistemi batch multiprogrammati	10
1.3.3	Sistemi time-sharing	10
1.4	Funzionamento Hardware	11
1.4.1	Interrupts hardware	11
1.4.2	Interrupts software (<i>traps</i>)	11
1.4.3	Handlers	11
1.4.4	Protezione delle risorse	11
1.5	Struttura di un Sistema Operativo	12
1.5.1	Processi	12
1.5.2	Gestione di processi	12
1.5.3	Memoria centrale	12
1.5.4	Gestione dei dispositivi di I/O	12
1.5.5	Gestione della memoria secondaria	12
1.5.6	Gestione del file system	12
1.5.7	Protezione e sicurezza	12
1.5.8	Interfaccia utente	13
1.5.9	Interfaccia programmatore	13
1.6	Struttura monolitica	13
1.7	Struttura modulare	13
1.8	Kernel	13
1.8.1	SO a Microkernel	14
1.8.2	SO a Kernel modulare	14
1.9	Macchine virtuali	14
1.9.1	Virtualizzazione	14
2	Processi e Thread	15
2.1	Concetto di processo	15
2.2	Stati di un processo	15
2.2.1	Process Control Block	16
2.2.2	Context Switching e PCB	16
2.3	Code di scheduling	16
2.3.1	Scheduler a breve termine (o di CPU)	16
2.4	Overhead	16
2.4.1	Overhead da scheduling	16
2.5	Operazioni sui processi	16
2.5.1	Creazione	17
2.5.2	Relazione padre-figlio	17
2.5.3	Terminazione	17
2.6	Processi leggeri (<i>Thread</i>)	17
2.7	Interazione tra processi	18
2.7.1	Interazione	18
2.7.2	Supporto all'interazione	18

3	Processi in UNIX	19
3.1	Stati di un processo UNIX	19
3.1.1	Processi swapped	19
3.2	Rappresentazione processi UNIX	19
3.2.1	Process Control Block	20
3.3	Immagine di un processo UNIX	20
3.4	System call per la gestione dei processi	21
3.4.1	Fork	21
3.4.2	Exit	22
3.4.3	Wait	22
3.4.4	Exec	23
3.4.5	Perror	24
3.5	Inizializzazione processi UNIX	24
3.6	Interazione con shell	25
4	Scheduling	26
4.1	CPU burst e I/O burst	26
4.2	CPU bound e I/O bound	26
4.3	Pre-emption	26
4.4	Politiche e meccanismi	26
4.5	Criteri di scheduling	26
4.6	Algoritmi di scheduling	26
4.6.1	FCFS (First-Come-First-Served)	26
4.6.2	SJF (Shortest-job-first)	27
4.7	Scheduling con priorità	27
4.7.1	Priorità	27
4.7.2	Problema & Soluzione	28
4.7.3	Round Robin	28
4.7.4	Approcci misti	28
4.8	Scheduling in UNIX	28
4.8.1	MLFQ	28
5	Interazione tra processi	29
5.1	Processi interagenti	29
5.2	Comunicazione	29
5.2.1	Scambio di messaggi	29
5.2.2	Meccanismi di comunicazione	29
5.2.3	Naming	29
5.2.4	Comunicazione diretta	29
5.2.5	Comunicazione indiretta	30
5.3	Buffering del canale	30
5.3.1	Send sincrona/asincrona	31
5.3.2	Send con sincronizzazioni estesa	31
5.4	Comunicazione in UNIX	31
5.4.1	pipe	31
5.5	Sincronizzazione	31
5.5.1	Mutua esclusione	31
5.5.2	Sincronizzazione tra processi nel modello ad ambiente globale	31
5.5.3	Sincronizzazione tra processi nel modello ad ambiente locale	31

6	Segnali in UNIX	32
6.1	Tutti i segnali	32
6.2	Gestione dei segnali	32
6.2.1	signal	32
6.2.2	kill	33
6.2.3	sleep	33
6.2.4	alarm	33
6.2.5	pause	34
6.3	Segnali e multiprogrammazione	34
6.3.1	fork	34
6.3.2	exec	34
6.4	Handlers	34
6.4.1	Affidabilità dei segnali	34
6.4.2	Interrompibilità delle system call	34
6.5	Segnali in Linux	35
7	File system	36
7.1	Organizzazione del file system	36
7.2	Struttura logica	36
7.2.1	File	36
7.2.2	Directory	36
7.3	Mounting	37
7.4	Operazioni sui file	37
7.5	Struttura interna del file	37
7.5.1	Blocchi	37
7.5.2	Record logici	37
7.6	Metodi d'accesso	37
7.6.1	Sequenziale	38
7.6.2	Diretto	38
7.6.3	A indice	38
7.7	Protezione	38
7.8	Organizzazione fisica	38
7.9	Metodi di allocazione	38
7.9.1	Allocazione contigua	38
7.9.2	Allocazione a lista concatenata	39
7.9.3	Tabella di allocazione dei file (FAT)	39
7.9.4	Allocazione ad indice	39
8	File system in UNIX	40
8.1	Organizzazione fisica	40
8.1.1	i-node	40
8.1.2	Indirizzamento	40
8.2	Protezione	41
8.2.1	Bit di protezione	41
8.3	Directory	41
8.4	Implementazioni del file system in UNIX	41
8.5	Accesso a file	42
8.5.1	File descriptor	42
8.5.2	Strutture dati e kernel	42
8.6	System call per l'accesso ai file	43
8.6.1	Open	43
8.6.2	Creat	44
8.6.3	Close	44
8.6.4	Read	45

8.6.5	Write	45
8.6.6	Lseek	46
8.7	System call per la gestione dei file	46
8.7.1	Unlink	46
8.7.2	Link	47
8.7.3	Access	47
8.7.4	Stat	48
8.7.5	Chmod	49
8.7.6	Chown	49
8.8	System call per la gestione dei direttori	49
8.8.1	Chdir	50
8.8.2	Opendir	50
8.8.3	Closedir	50
8.8.4	Readdir	51
8.8.5	Mkdir	51
8.9	Comunicazione tra i processi in UNIX	51
8.9.1	Pipe	52
8.9.2	FIFO	53
8.10	System call per la comunicazione	53
8.10.1	pipe	53
8.10.2	mkfifo	54
8.11	Ridirezione di I/O e Piping	54
8.11.1	Ridirezione in output	54
8.11.2	Ridirezione in input	54
8.11.3	Piping di comandi	54
8.11.4	Dup	55
9	Gestione della Memoria	56
9.1	Compiti del Sistema Operativo	56
9.2	Spazio di Indirizzamento	56
9.2.1	Esecuzione	56
9.2.2	Indirizzi logici e fisici	56
9.2.3	Binding	57
9.2.4	Tecniche di allocazione su memoria centrale	57
9.3	Allocazione contigua a partizione singola	57
9.4	Allocazione contigua a partizione multipla	57
9.4.1	Protezione	58
9.5	Allocazione non contigua con paginazione	58
9.5.1	Vantaggi	59
9.5.2	Supporto hardware	59
9.5.3	Tabella delle pagine	59
9.5.4	Translation Look-aside Buffers (TLB)	60
9.5.5	Protezione	60
9.5.6	Paginazione a più livelli	60
9.5.7	Tabella delle pagine invertita	61
9.6	Segmentazione	61
9.6.1	Struttura	61
9.6.2	Supporto hardware	61
9.6.3	Segmentazione paginata	62
9.6.4	Segmentazione in Linux	62
9.7	Overlay	62
9.8	Memoria virtuale	62
9.8.1	Vantaggi	62
9.9	Paginazione su richiesta	62

9.9.1	Pager	63
9.9.2	Funzionamento	63
9.9.3	Page Fault	63
9.9.4	Sovrallocazione	63
9.9.5	Dirty Bit	64
9.9.6	Implementazione algoritmo LRU	64
9.9.7	Thrashing	64
9.10	Working Set	64
9.10.1	Funzionamento	65
9.10.2	Valore di Δ	65
9.11	UNIX	65
9.11.1	Versioni iniziali	65
9.11.2	Swapping	65
9.11.3	Versioni attuali	66
9.11.4	Algoritmo di sostituzione	66
9.11.5	Scheduling, paginazione e swapping	66
9.12	Linux	67
9.12.1	Organizzazione della memoria fisica	67
9.12.2	Spazio di indirizzamento	67
9.12.3	Paginazione	67
9.12.4	Page-fetching e sostituzione	67
9.13	MS Windows XP	67
10	File Comandi in UNIX	68
10.1	Shell	68
10.1.1	Molteplici shell	68
10.1.2	Shell di login	68
10.1.3	Comandi	68
10.1.4	Logout	68
10.1.5	Comandi	68
10.2	Comandi shell Linux	68
10.2.1	Piping	69
10.2.2	Espressioni	69
10.3	Metacaratteri	69
10.4	Variabili	69
10.4.1	Ambiente	69
10.5	Espansione	69
10.5.1	Sequenza	69
10.6	Shell scripting	70
10.6.1	Esecuzione	70
10.6.2	Argomenti	70
10.6.3	Variabili notevoli	70
10.6.4	Input e output	70
10.7	Test	70
10.7.1	Parametri più comuni	71
10.7.2	Alternativa	71
10.8	Strutture di controllo	71
10.8.1	If	71
10.8.2	Alternativa multipla	71
10.8.3	Cicli enumerativi	72
10.8.4	Cicli non enumerativi	72

11 Programmazione Concorrente in modello ad ambiente globale	73
11.1 Mutua esclusione	73
11.2 Operazioni indivisibili	73
11.3 Sezione critica	73
11.4 Deadlock	73
11.4.1 Grafi di allocazione delle risorse	74
11.4.2 Condizioni necessarie	74
11.4.3 Trattamento	74
11.5 Soluzioni al problema della mutua esclusione	75
11.5.1 Struttura generale	75
11.5.2 Possibili soluzioni	75
11.6 Soluzioni algoritmiche	75
11.6.1 Algoritmo di Dekker	75
11.6.2 Algoritmo di Peterson	76
11.7 Attesa attiva	76
11.7.1 Alteriori requisiti	76
11.8 Soluzione hardware	76
11.8.1 Disabilitazione interrupt	76
11.8.2 Test and Set	77
11.8.3 Lock e Unlock	77
11.9 Soluzioni software	78
11.10 Semafori	78
11.10.1 Implementazione	78
11.10.2 Atomicità p e v	79
11.10.3 Casi d'uso	79
11.10.4 Problemi	79
12 Java thread (1)	80
12.1 Implementazione	80
12.1.1 Thread	80
12.1.2 Runnable	81
12.2 Sincronizzazione di thread	81
12.2.1 Join	81
12.2.2 Synchronized	81
12.2.3 Semafori	82
12.3 Stop	82
12.4 Suspended	82
12.5 Ulteriori metodi dei thread	82
13 Monitor	83
13.1 Utilità	83
13.2 Sincronizzazione	83
13.2.1 Implementazione	83
13.3 Variabili condizione	83
13.3.1 wait	83
13.3.2 signal	83
13.4 Semantiche dell'operazione signal	84
13.4.1 signal_and_urgent_wait	84
13.4.2 signal_and_return	84
13.4.3 signal_all	84

14 Java thread (2)	85
14.1 Variabili condizione	85
14.1.1 Lock	85
14.1.2 Utilizzo	85
14.2 Monitor	85
14.3 Problema dei filosofi a cena	85
14.3.1 Definizione	85
14.3.2 Soluzione problematica	86
14.3.3 Soluzione corretta	86
15 Gestione della I/O	87
15.1 Sottosistema di I/O	87
15.1.1 Nascondere i dettagli	87
15.1.2 Omogenizzazione	87
15.1.3 Malfunzionamenti	87
15.1.4 Spazio dei nomi	87
15.1.5 Sincronizzazione	87
15.2 Architettura	88
15.3 Buffering	88
15.4 Gestione malfunzionamenti	88
15.4.1 Gestione	88
15.5 Allocazione dei dispositivi	88
15.6 Livello dipendente dai dispositivi	89
15.6.1 Controllore di un dispositivo di I/O	89
15.7 Modalità di gestione	89
15.7.1 Gestione a controllo di programma	89
15.7.2 Gestione basata su interruzioni	89
15.8 Astrazione di un dispositivo	90
15.8.1 Descrittore di un dispositivo	90
15.9 Gestione di dispositivi speciali: il temporizzatore	91
15.9.1 Clock	91
15.9.2 Controllore	91
15.10 Gestione e organizzazione dei dischi	91
15.10.1 Indirizzo di un settore	91
15.10.2 Scheduling delle richieste di trasferimento	92
15.10.3 Politiche di scheduling delle richieste	92

1 Introduzione

Per *Sistema Operativo (SO)* si intende un programma, o insieme di programmi, che agisce da intermediario tra utente e *hardware*. Esso si preoccupa della gestione delle risorse, accesso all'*hardware* e creazione di un ambiente sicuro per lo sviluppo di software.

1.1 Hardware

Quando parliamo di *hardware* del quale si occupa il SO ci riferiamo ai componenti di un computer, quali, ad esempio:

- CPU
- RAM
- Memoria di massa
- Dispositivi di I/O
- Dispositivi di rete
- etc.

successivamente mappati in risorse *logiche*:

- CPU → Processi
- RAM → Memoria Virtuale
- Memoria di massa → File System
- Dispositivi di I/O → Driver

1.2 Aspetti che differenziano i sistemi operativi

Tutti i sistemi operativi si differenziano nei seguenti aspetti, che ne contraddistinguono qualità e utilizzabilità:

- Architettura
- Condivisione (delle risorse)
- Efficienza
- Estendibilità
- Protezione/Sicurezza
- Affidabilità/Tolleranza a guasti
- Conformità e standard

1.3 Sviluppi storici

Negli anni '50 nasce la prima generazione di computer. Essa aveva un controllo dello stato manuale, nonché un funzionamento meccanico basato su valvole termioniche. Questi primi sistemi, oggi considerati arcaici, non presentavano un SO.

Tra gli anni '55 e '65 si sviluppa la capacità di scrivere (in un linguaggio di alto livello) dei programmi (*job*). Questi venivano interpretati da una macchina con l'utilizzo di schede perforate, aggregate ad altri programmi con esigenze simili in lotti (*batch*) ed eseguiti in maniera sequenziale. Nascono qui i primi SO, con il compito di trasferire il controllo tra *job* terminato e successivo. Questi erano residenti in memoria (*Monitor*), mancavano di interazione con l'utente e non sfruttavano l'inattività della CPU nelle fasi di I/O.

1.3.1 Sistemi batch Semplici

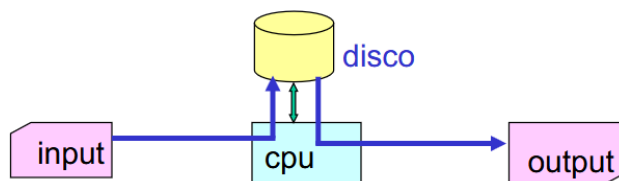
Nei sistemi *batch semplici* viene definito un sistema operativo persistente all'interno della memoria centrale, a sua volta contenente al più un solo *job* contemporaneamente.

Questi sistemi non permettevano l'esecuzione di un nuovo *job* fino alla terminazione del precedente, l'utilizzo della CPU in attesa della I/O, interazione con l'utente e sono generalmente lenti ad accedere alle risorse.

1.3.2 Sistemi batch multiprogrammati

Viste le limitazioni dei sistemi *batch semplici* nasce il concetto di *spooling*, ovvero l'utilizzo di un buffer (un disco esterno) al fine di memorizzarne all'interno i programmi da eseguire (*spool*), leggere in anticipo i dati, memorizzare temporaneamente i risultati e poter facilitare lo scambio di *jobs*.

In particolare, il SO seleziona un sottoinsieme di *jobs* appartenenti al *pool* da caricare nella memoria centrale. Di questi, poi, ne sceglie uno (*job corrente*) a cui assegnare la CPU. Se il processo si pone in attesa di un evento allora viene temporaneamente sostituito con un altro *job*

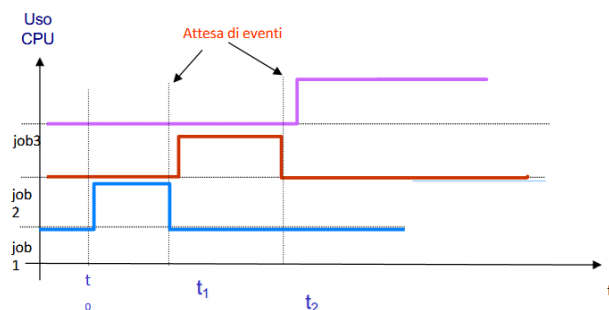


Il SO è quindi in grado di portare avanti l'esecuzione di più *job* contemporaneamente, in particolare:

- Ad ogni istante
 - Un solo *job* utilizza la CPU
 - Gli altri *jobs* appartenenti al *pool* attendono
- Quando il *job* attivo si pone in attesa di un evento
 - Il SO decide a quale nuovo *job* assegnare la CPU (*scheduling*)
 - Viene effettuato il cambio di contesto (*context switching*)

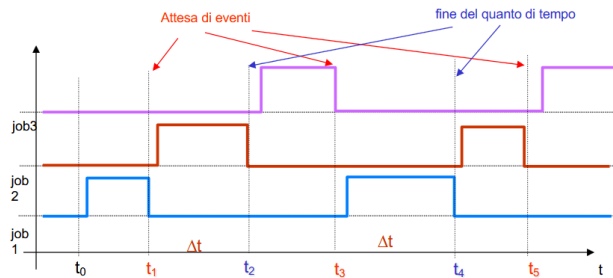
Context Switching Consiste nel passaggio dall'esecuzione di un *job* ad un altro. In particolare, si salvano i valori dei registri della CPU relativi al primo *job* all'interno della memoria e si caricano i valori precedentemente salvati, se presenti, relativi al secondo *job*.

Scheduling Con *scheduling* si intende la scelta, da parte del SO, del processo al quale assegnare l'utilizzo della CPU (*short-term scheduling*) e dei processi da caricare sulla memoria centrale (*long-term scheduling*).



1.3.3 Sistemi time-sharing

Dalle necessità di aggiungere interazione con l'utente e permettere l'esistenza di più utenti contemporanei, nascono i sistemi *time-sharing*. Quest'ultimi presentavano per ogni utente una macchina virtuale indipendente e implementavano una nuova politica di *scheduling* che eseguiva un *context switching* ad intervalli regolari prefissati (*time slice*) e forniva una percezione di esecuzione contemporanea a tutte le macchine virtuali.



È essenziale che questi sistemi si preoccupino anche della separazione della memoria tra i *job*, ai quali viene ora assegnata una più ridotta porzione della memoria centrale (detta *memoria virtuale*), e della sincronizzazione e comunicazione dei *job* stessi.

1.4 Funzionamento Hardware

La comunicazione tra SO e componenti *hardware**software* avviene mediante l'utilizzo di *interrupt*. Ognuno di questi è associato ad un evento e viene gestito mediante un *handler*

1.4.1 Interrupts hardware

I dispositivi mandano dei segnali alla CPU per notificarla della presenza di un evento o del cambiamento di uno stato.

1.4.2 Interrupts software (*traps*)

I software tentano l'esecuzione di operazioni illecite oppure richiedono l'esecuzione di una *system call*

1.4.3 Handlers

Quando un SO riceve un'interruzione deve:

1. Interrompere l'esecuzione
2. Salvare lo stato in memoria
3. Ricavare la *routine* da eseguire tramite il *vettore delle interruzioni*
4. Attivare la routine
5. Ripristinare lo stato precedente

1.4.4 Protezione delle risorse

All'interno di sistemi *time-sharing* è importante la presenza di meccanismi di protezione (*hardware* e *software*) delle risorse che non permettano a programmi e utenti di accedere in maniera illecita a risorse assegnate ad altri programmi/utenti.

Memoria Dal punto di vista della memoria, ad esempio, è importante fare in modo che un qualunque *job* non possa accedere al di fuori della porzione di memoria centrale a lui assegnata.

CPU La protezione della CPU, o meglio di un sottoinsieme di istruzioni "pericolose" del suo *instruction set*, viene implementata mediante la creazione di *ring* di sicurezza:

- User mode
- Kernel mode

La modalità utilizzata in un determinato momento viene definita attraverso un bit all'interno del processore stesso e limita l'esecuzione di alcune istruzioni (dette *istruzioni privilegiate*). Quest'ultime possono essere eseguite solamente in *kernel mode* e, teoricamente, solo da SO. Tutti i programmi utente, invece, si trovano in *user mode* e, quindi, con un accesso ristretto alle operazioni. Se un processo dovesse tentare l'utilizzo di un'istruzione a lui non disponibile è la CPU stessa a bloccarne l'esecuzione e a generare un *interrupt*.

System call Può essere talvolta utile ai programmi utente utilizzare le *istruzioni privilegiate*. Il SO, allora, offre delle API dette *system call* che non fanno altro che permettere di delegare al SO, da parte dei *job*, l'esecuzione in *kernel mode* le operazioni che, altrimenti, non potrebbero eseguire. In particolare:

1. Chiamata a *system call*
2. Invio *interrupt* software
3. Salvataggio dello stato
4. Esecuzione dal parte del SO in modalità *kernel* della/e operazioni
5. Ripristini stato ed esecuzione ripresa dal chiamante

1.5 Struttura di un Sistema Operativo

1.5.1 Processi

Per processi si intendono i programmi caricati in memoria. Essi si definiscono "entità attive", sono cioè unità di esecuzione all'interno del SO, e rappresentano l'istanza di un programma con il relativo contesto di esecuzione. I programmi, invece, si definiscono "entità passive", essendo semplicemente un'unione di byte rappresentanti le istruzioni da eseguire.

1.5.2 Gestione di processi

Più processi possono essere in esecuzione contemporaneamente in un sistema ed è compito del SO gestirne: creazione, terminazione, sospensione, ripristino, sincronizzazione e comunicazione.

1.5.3 Memoria centrale

È importante che un SO gestisca: separazione della memoria associata ai processi, allocazione e deallocazione della memoria e, più in generale, la memoria virtuale associata ai processi.

1.5.4 Gestione dei dispositivi di I/O

Il SO deve gestire l'interfaccia tra i programmi ed i dispositivi del sistema e, per ogni dispositivo, deve predisporre un *device driver*, ovvero una routine per la gestione del dispositivo che contenga intrinsecamente la conoscenza specifica dello stesso.

1.5.5 Gestione della memoria secondaria

Così come per i dispositivi di I/O, un SO si occupa di tutte le operazioni sulla memoria secondaria: rappresentazione logica dello spazio in blocchi, allocazione/deallocazione dei blocchi, gestione dello spazio e *scheduling* delle operazioni sul disco.

1.5.6 Gestione del file system

Con il *file system* il SO si preoccupa di rappresentare la/e memoria/e secondaria/e del sistema, fornendo una visione logica uniforme. Realizza il concetto astratto di file (come singola unità di memoria) e di directory (come organizzatore di file) con tutte le possibili relative operazioni.

1.5.7 Protezione e sicurezza

È compito del SO assicurare protezione, ovvero controllo sull'accesso alle risorse (mediante autorizzazioni e modalità di accesso), e sicurezza, ovvero l'affidabilità rispetto agli accessi indesiderati.

1.5.8 Interfaccia utente

È sempre presentata dal SO almeno un'interfaccia che permette all'utente di comunicare ed interagire con i componenti del sistema. Essa può essere di tipo:

- Comandi (*shell*): mediante una linea di comando
- Grafica (*GUI*): mediante l'interazione con elementi grafici sullo schermo

1.5.9 Interfaccia programmatore

Con l'*interfaccia programmatore* (API) il SO si preoccupa di fornire all'utente una serie di operazioni attraverso l'utilizzo di *system call*. Si definisce *programma di sistema* un programma che fa utilizzo di *system call*.

1.6 Struttura monolitica

Una possibile struttura di SO consiste in un unico modulo eseguibile contenente una serie di procedure che realizzano i vari componenti. L'interazione tra i componenti viene implementata attraverso il meccanismo di chiamata a procedura.

Il vantaggio di una struttura di questo tipo è il basso costo di interazione tra i componenti, a patto, però, di una più complessa manutenzione del sistema.

1.7 Struttura modulare

Il SO è formato da una serie di moduli, rappresentanti i componenti, caratterizzati da interfacce ben definite. Il sistema è quindi formato da una serie di livelli sovrapposti, ognuno dei quali utilizza i livelli inferiori e viene utilizzato dai superiori.

livello 5: programmi di utente
livello 4: buffering dei dispositivi di I/O
livello 3: driver della console
livello 2: gestione della memoria
livello 1: scheduling CPU
livello 0: hardware

Una struttura di questo tipo risulta molto vantaggiosa dal punto di vista dell'astrazione (essendo ogni livello un oggetto astratto) e della modularità. Allo stesso tempo, però, si dimostra di difficile realizzazione e scarsa efficienza.

1.8 Kernel

Con *Kernel* si intende la parte più "profonda" del sistema operativo, quella incaricata di interfacciarsi con l'hardware.

1.8.1 SO a Microkernel

Un SO a *Microkernel* è caratterizzato da una struttura semplice e ridotta a poche funzionalità di base. Il resto del sistema è dato dai processi utente.

Un sistema di questo tipo è particolarmente affidabile, estendibile e poco efficiente (visto l'ampio utilizzo di *system call*)

1.8.2 SO a Kernel modulare

Un'ulteriore possibile struttura per un *kernel* è quella modulare. Essa è rappresentabile secondo tecniche *object-oriented* ed è facilmente estendibile. Ogni modulo è separato dagli altri, comunica utilizzando interfacce note ed è facilmente caricabile all'interno del sistema ove necessario.

1.9 Macchine virtuali

Le *macchine virtuali* sono strumenti che permettono la *virtualizzazione* dell'hardware e del *kernel* di una macchina. Creano l'illusione dell'esistenza di più macchine su una stessa macchina ospitante, ognuna delle quali con un proprio hardware.

1.9.1 Virtualizzazione

Con *virtualizzazione* si intende la presentazione all'utente di una struttura *hardware* e *software* differente da quella realmente presente all'interno della macchina. Questo processo è ottenibile aggiungendo un ulteriore livello intermedio al sistema, tra visione logica e fisica.

L'obiettivo ultimo di una tecnica di questo tipo è il disaccoppiamento del comportamento tra le risorse visualizzate dall'utente e quelle effettivamente presenti nella macchina.

I *software* che realizzano la virtualizzazione sono detti *Virtual Machine Monitor* (VMM).

VMM di Sistema Le funzionalità di virtualizzazione fanno parte di un SO leggero, costituendo un unico sistema posto direttamente sopra l'*hardware*.

VMM Ospitanti La VMM è rappresentata da un software che viene installato sopra il SO ed accede all'hardware tramite le *system call* che il SO fornisce.

2 Processi e Thread

2.1 Concetto di processo

Un processo è un programma in esecuzione. Esso è composto da:

- Codice (text): programma eseguito
- Dati: variabili globali
- *Program Counter*
- Registri CPU
- *Stack*

Ad un processo possono essere, poi, associate le risorse aperte come:

- File aperti
- Connessioni di rete utilizzate
- Dispositivi in uso
- etc

2.2 Stati di un processo

Durante la sua vita, un processo può trovarsi in molteplici stati:

Init

Il processo viene caricato in memoria dal SO e il SO ne inizializza i dati

Ready

Il processo è pronto per essere eseguito dalla CPU

Running

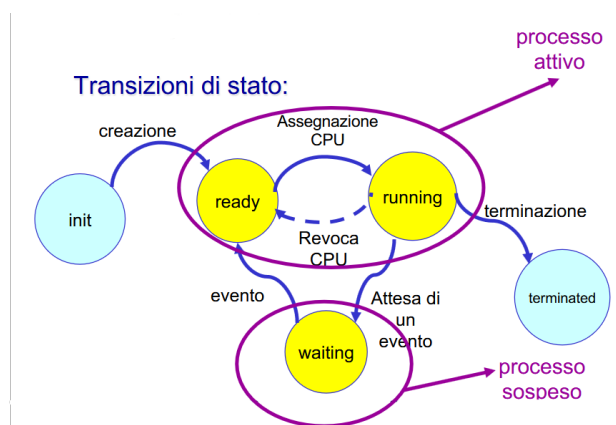
Il processo sta utilizzando la CPU

Waiting

Il processo è sospeso in attesa di un evento

Terminated

Il processo è terminato ed è in fase di deallocazione dalla memoria



In un sistema monoprocesso e multiprogrammato al più un processo può essere attivo contemporaneamente. È necessario, quindi, che esistano delle strutture in memoria che mantengano le informazioni relative ai processi in stato di attesa (*ready* e *waiting*).

2.2.1 Process Control Block

Ad ogni processo viene associato un *Process Control Block* (PCB) che contiene tutte le informazioni relative al processo:

- Stato del processo
- Contenuto dei registri
- Informazioni per gestire la memoria (registri, limite, ...)
- Informazione relativa all'I/O (risorse allocate, file aperti, ...)
- Informazioni sullo scheduling (priorità, puntatori a code, ...)
- Informazioni sull'accounting (tempo CPU utilizzato, ...)
- etc.

È compito del SO gestire i PCB di ogni singolo processo attivo.

2.2.2 Context Switching e PCB

È nelle fasi di *Context Switching* che il sistema operativo si preoccupa di salvare lo stato del processo interrotto all'interno del relativo PCB e ripristinare lo stato del processo in avvio dal relativo PCB.

2.3 Code di scheduling

Vengono chiamate *code di scheduling* quelle code gestite dal SO che contengono tutti i processi in un determinato stato di attesa. Queste possono essere gestite con molteplici politiche, a discrezione del SO stesso, e possono riferirsi al desiderio di un processo di utilizzare la CPU, così come di utilizzare un dispositivo di I/O.

2.3.1 Scheduler a breve termine (o di CPU)

Con *scheduler a breve termine* si intende la gestione della coda di processi in stato di *ready*.

2.4 Overhead

Si indica con *overhead* il tempo di CPU impiegato dal SO per eseguire i propri compiti. Durante questo periodo di tempo la CPU non può essere utilizzata da alcun processo.

$$\text{Overhead} = \frac{\text{Tempo}_{\text{SO}} \times 100}{\text{Tempo}_{\text{Utente}}}$$

2.4.1 Overhead da scheduling

Anche le operazioni di *scheduling* vanno ad impattare sull'*overhead* di un SO. I fattori che ne modificano sostanzialmente la durata sono:

- Frequenza del cambio di contesto
- Costo dei trasferimenti da/verso memoria
- Dimensione del PCB

Esistono SO che prevedono l'esistenza di processi "leggeri" (*thread*) che hanno la particolare caratteristica di condividere codice e dati con altri *thread*. In questo caso è quindi possibile ridurre la grandezza del PCB.

2.5 Operazioni sui processi

Ogni SO multiprogrammato mette a disposizione delle *system call* che permettano all'utente di gestire i processi.

2.5.1 Creazione

Un processo "padre" può richiedere la creazione di un processo "figlio".

2.5.2 Relazione padre-figlio

La relazione tra processi padre e figli si divide principalmente in tre aspetti:

- Concorrenza
 - Padre e figlio procedono contemporaneamente
 - Padre genera uno o più figli concorrenti e attende la terminazione
 - etc.
- Spazio degli indirizzi
 - Condiviso tra padre e figli
 - Separato tra padre e figli
- Condivisione delle risorse
 - Le risorse del padre vengono condivise anche con i figlio
 - Il figlio deve esplicitamente fare richiesta delle risorse
 - etc.

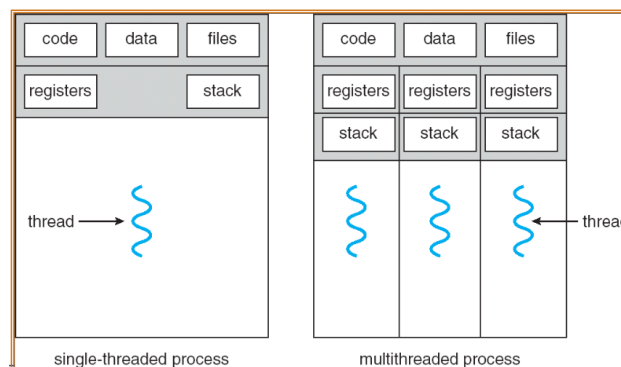
2.5.3 Terminazione

Ogni processo può essere sia figlio di un altro processo che padre di più processi. Queste informazioni devono essere salvate dal SO nei descrittori.

Qualora in processo dovesse terminare, è compito del SO notificare al padre l'evento e gestire l'uccisione o l'azione dei figli.

2.6 Processi leggeri (*Thread*)

Un *thread* è un'unità d'esecuzione che condivide codice e dati con gli altri *thread* associati allo stesso *task*. Ogni *thread* è caratterizzato, quindi, unicamente da *Program Counter*, registri e stack.

$$\text{Thread} = \{\text{Program Counter, Registri, Stack, ...}\}$$
$$\text{Task} = \{\text{thread}_1, \text{thread}_2, \dots, \text{thread}_n, \text{text, dati, risorse}\}$$


I vantaggi legati a questo tipo di processi sono la condivisione delle variabili tra *thread* e, generalmente, un minor costo per il *context switching*. Lo svantaggio di un'architettura di questo tipo, invece, è la poca sicurezza tecnica legata alla condivisione dei dati (e quindi l'implementazione di necessarie tecniche di sincronizzazione).

2.7 Interazione tra processi

È possibile classificare una coppia di processi in due gruppi:

- Indipendenti: due processi non si influenzano durante l'esecuzione
- Dipendenti: due processi si influenzano durante l'esecuzione

Nel caso di processi interagenti nasce la necessità di sviluppare tecniche di interazione.

2.7.1 Interazione

Cooperazione È un'interazione prevedibile e desiderata. I due processi sono intrinsecamente pensati per collaborare e cooperano per il raggiungimento di un obiettivo comune.

Competizione Interazione prevedibile ma non desiderata. Due processi si sincronizzano per l'accesso a una risorsa comune.

Interferenza Unica interferenza non prevedibile e non desiderata. Potenzialmente dannosa.

2.7.2 Supporto all'interazione

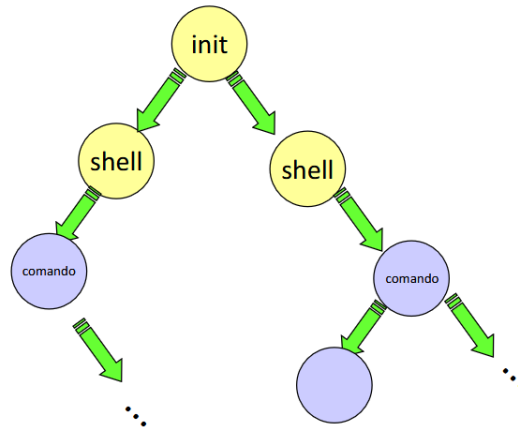
Memoria condivisa (ambiente globale) È consentita la condivisione di variabili tra i *thread* ed il SO fornisce dei meccanismi che permettono la sincronizzazione sull'accesso a suddette variabili.

Scambio di messaggi (ambiente locale) I processi non condividono variabili e interagiscono mediante meccanismi di messaggistica forniti dal SO.

3 Processi in UNIX

UNIX è un SO multiprogrammato a divisione di tempo. L'unità di computazione è il processo "pesante", con dati non condivisibili e codice rientrante.

Ogni processo ha un proprio spazio di indirizzamento locale e non condiviso ed il sistema è caratterizzato dalla seguente gerarchia:



3.1 Stati di un processo UNIX

All'interno dei sistemi UNIX vengono aggiunti 2 ulteriori possibili stati per ogni processo:

Zombie

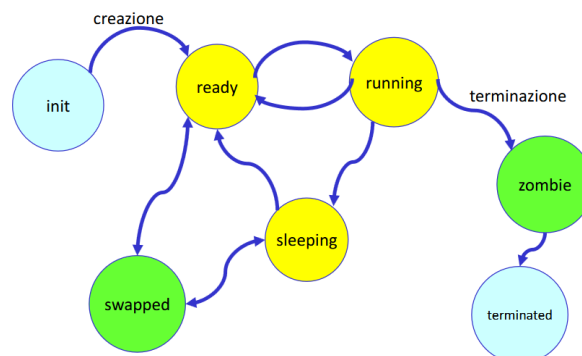
Il processo è terminato ed in attesa che il padre ne rilevi lo stato

Swapped

Il processo è temporaneamente trasferito in memoria secondaria

3.1.1 Processi swapped

Lo *scheduler a medio termine* si occupa dello swap dei processi tra memoria centrale e secondaria. In particolare, effettua uno *swap out* verso la memoria secondaria ai processi bloccati e uno *swap in* verso la memoria centrale ai processi che possono riprendere.



3.2 Rappresentazione processi UNIX

Il codice dei processi UNIX è detto rientrante, ovvero possibilmente condiviso tra più processi. Il SO, infatti, gestisce una struttura globale detta *text table*, al cui interno sono contenuti tutti i puntatori ai codici utilizzati.

Ogni riga all'interno della *text table* è detta *text structure* e contiene, oltre al puntatore al codice, il numero di processi a cui è associata.

3.2.1 Process Control Block

Ogni PCB in UNIX è composto da due componenti:

Process structure Informazioni necessarie al sistema per la gestione del processo come:

- Stato del processo
- Puntatore a dati e stack
- Riferimento alla *text structure* associata
- Informazioni di scheduling
- Riferimento al processo padre
- Informazioni relative ai segnali
- Puntatore al processo successivo nella coda dei processi ai quali appartiene
- Riferimento alla *user structure*

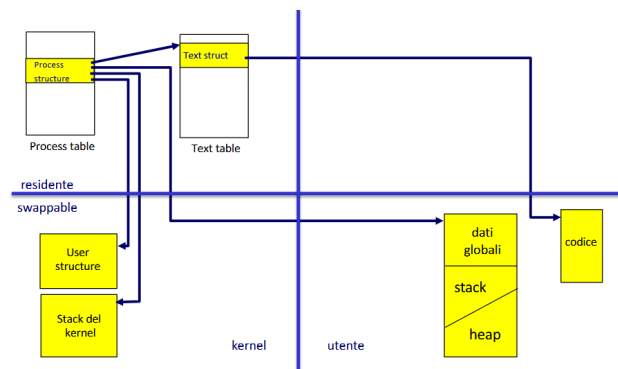
Ogni *process structure* è conservata dal SO all'interno della *process table*.

User structure Informazione necessaria al sistema se il processo si trova in memoria centrale:

- Valore dei registri
- Informazioni su risorse allocate
- Informazioni sulla gestione dei segnali
- Ambiente del processo

3.3 Immagine di un processo UNIX

Con "immagine di un processo" si intende l'insieme delle aree di memoria e strutture dati associate al processo. Non tutta l'immagine è accessibile dall'utente in *user mode* e non tutta l'immagine può essere *swappable* quando il processo è soggetto a *swapping*.



I componenti dell'immagine di un processo UNIX sono:

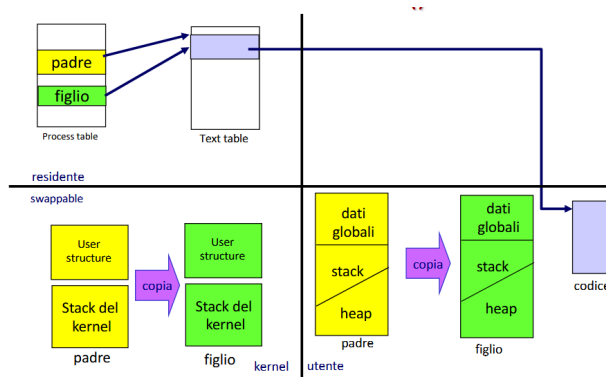
- Process structure
- User structure
- Elemento della *text table* associato al processo
- Area di codice
- Area dati globali dell'utente
- Stack e heap
- Stack del kernel

3.4 System call per la gestione dei processi

3.4.1 Fork

Nome	fork
Descrizione	Permette ad un processo di generare un figlio. Entrambi condividono lo stesso codice ed il figlio eredita una copia dei dati del padre.
Parametri	La funzione non ha parametri in ingresso.
Valore di ritorno	int – PID del processo figlio nel padre, 0 nel figlio e un valore negativo in caso di errore
Libreria	unistd.h
Firma	<pre>#include <unistd.h> pid_t fork(void);</pre>

Effetti Quando chiamata, una *fork* alloca una nuova *process structure* per il processo figlio e alloca una nuova *user structure* copia di quella del padre. Vengono poi create una nuova area di dati e *stack* (sempre copiando quella del processo chiamante) e viene, infine, aggiornato il riferimento al *text*.



Relazione padre-figlio In relazione ad una *fork*, padre e figlio procedono contemporaneamente lo stesso codice. Lo spazio degli indirizzi è duplicato, ogni variabile del figlio assume lo stesso valore che aveva nel padre nel momento della *fork* ed infine la *user struct* viene duplicata quindi le risorse diventano condivise e gli *handler* vengono copiati, così come il *program counter*.

3.4.2 Exit

Nome	exit
Descrizione	Permette ad un processo di terminare la propria esecuzione comunicando al padre un'informazione numerica.
Parametri	status – intero arbitrario utilizzabile per la comunicazione
Valore di ritorno	void
Libreria	unistd.h
Firma	<pre>#include <unistd.h> void exit(int status);</pre>

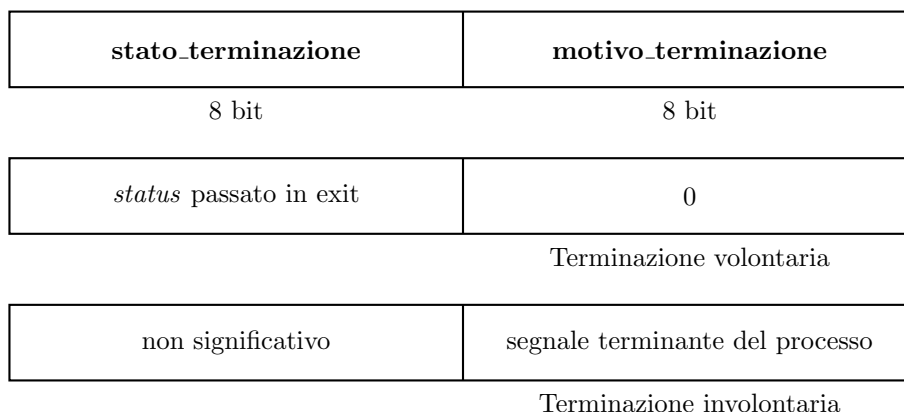
Effetti A seguito di una *exit* vengono chiusi tutti i file non condivisi associati al processo e tutti i figli vengono "adottati" dal processo *init*. Se il processo ha un padre allora rimane in stato *zombie* finchè quest'ultimo non ne rileva la terminazione.

Tutti i processi figli di *init* non possono andare in stato *zombie*.

3.4.3 Wait

Nome	wait
Descrizione	Permette ad un processo di rilevare lo stato di terminazione di un figlio. Bloccante se non sono presenti figli in stato <i>zombie</i> . Non bloccante se presenti figli in stato <i>zombie</i> o in caso di mancata presenza di figli.
Parametri	*status – indirizzo della variabile contenente lo stato di terminazione del figlio
Valore di ritorno	int – PID del processo figlio terminato, oppure -1 in caso di errore
Libreria	sys/wait.h
Firma	<pre>#include <sys/wait.h> int wait(int *status);</pre>

Risultato Lo *status* ottenibile dalla funzione *wait* utilizza la seguente struttura (ipotesi intero a 16 bit):



Rilevazione dello stato Al fine di semplificare la rivelazione dello stato risultante della *wait* vengono fornite 2 macro:

- **WIFEXITED(status)**: vero se il processo è terminato volontariamente. Con **WEXITSTATUS(status)** è possibile ricavare lo *status* effettivo
- **WIFSIGNALED(status)**: vero se il processo è terminato involontariamente. Con **WTERMSIG(status)** è possibile ricavare il segnale che ha causato la terminazione

3.4.4 Exec

La famiglia di funzioni *exec* permette di lanciare un processo da un processo ma si differenzia sostanzialmente dalle *fork*. Attraverso l'utilizzo di una funzione di tipo *exec*, il codice del nuovo processo non è il duplicato del codice del chiamante poiché non viene generato un figlio, bensì è il chiamante stesso a ricevere la modifica. Di fatto, se si lancia un programma attraverso una *exec* è il processo chiamante a diventare il processo per il programma chiamato.

Effetti Successivamente ad una *exec* il processo mantiene la stessa *process structure* (ad eccezione delle informazioni relative al codice), ha codice, dati globali, *stack* e *heap* nuovi, si riferisce a un nuovo *text* e mantiene la stessa *user structure* (ad eccezione del *program counter*)

Varianti Le possibili varianti delle *exec* si differenziano dal suffisso:

l: gli argomenti vengono specificati tramite una lista terminata da `(char)*0`

p: il file viene ricercato dalla *exec* nel *path*

v: il gli argomenti vengono specificati tramite un vettore

e: viene fornito un vettore che sostituisce l'*environment* corrente

Exec	Nome	execl
	Descrizione	Sostituisce il processo chiamante con quello specificato
	Parametri	pathname – nome dell'eseguibile da caricare arg0, ..., argN – argomenti per il nuovo processo (char *)0 – terminatore
	Valore di ritorno	void
	Libreria	unistd.h
	Firma	<pre>#include <unistd.h> int execl(const char *pathname, const char * arg0, ..., const char *argN, (char *)0);</pre>

Execve	Nome	execl
	Descrizione	Sostituisce il processo chiamante con quello specificato
	Parametri	pathname – nome dell'eseguibile da caricare *argv[] – argomenti per il nuovo processo *env[] – variabili d'ambiente del nuovo processo
	Valore di ritorno	void
	Libreria	unistd.h
	Firma	<pre>#include <unistd.h> int execl(const char *pathname, char *argv[], char *env[]);</pre>

3.4.5 Perror

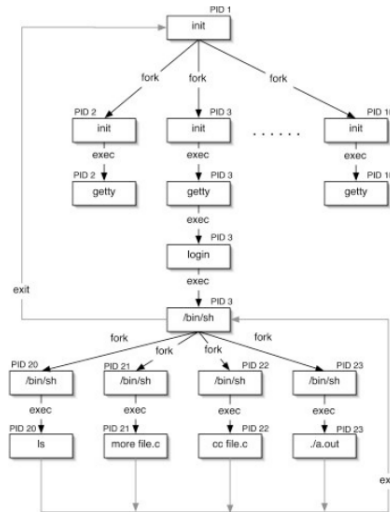
	Nome	perror
	Descrizione	Stampa, preceduto da una stringa fornita, l'ultimo errore ricavabile da <i>errno</i>
	Parametri	string – stringa prefisso da stampare
	Valore di ritorno	void
	Libreria	stdio.h
	Firma	<pre>#include <stdio.h> void perror(char *string);</pre>

errno Il valore stampato da *perror* è quello contenuto in *errno*. Quest'ultima è una variabile alla quale il *kernel* assegna un valore simbolico quando si verifica un errore durante una *system call*. Il significato di ogni possibile valore è consultabile in `sys/errno.h`.

3.5 Inizializzazione processi UNIX

In UNIX, l'inizializzazione di un processo segue il seguente schema:

1. *init* genera un processo per ogni terminale collegato - comando *getty*
2. *getty* controlla l'accesso al sistema mediante il comando *login*
3. In caso di accesso corretto, *login* esegue lo *shell* (specificato in `/etc/passwd`)



3.6 Interazione con shell

Ogni utente ha la possibilità di comunicare con la *shell* attraverso l'invocazione di comandi. Ogni comando è un eseguibile presente all'interno del *file system* (generalmente all'interno della cartella */bin*). Per ogni comando, la *shell* genera un processo figlio dedicato all'esecuzione di un comando, che può dividersi in 2 tipi:

- Foreground: la *shell* si pone in attesa del figlio

```
ls pippo -l
```

- Background: la *shell* continua l'esecuzione contemporaneamente al figlio

```
ls pippo -l &
```

4 Scheduling

Lo *scheduling* nasce come risultato della multiprogrammazione, al fine di massimizzare l'utilizzo della CPU.

4.1 CPU burst e I/O burst

Ogni processo alterna fasi in cui utilizza unicamente la CPU (*CPU burst*) a periodi nei quali comunica con dispositivi I/O (*I/O burst*).

Nelle fasi di *I/O burst* un processo non utilizza la CPU, che, in sistemi multiprogrammati, permette allo *short-term scheduler* di assegnarla a un nuovo processo.

4.2 CPU bound e I/O bound

Un processo che in prevalenza utilizza attività I/O è detto *I/O bound*. Al contrario, un processo che utilizza maggiormente la CPU è detto *CPU bound*.

4.3 Pre-emption

È possibile classificare gli algoritmi di *scheduling* in due categorie:

- Non *pre-emptive*: la CPU rimane allocata al processo finché quest'ultimo non termina oppure non la rilascia volontariamente
- *Pre-emptive*: il processo può essere "prelazionato", ovvero il SO può sottrargli la CPU

I sistemi a divisione di tempo adottano sempre politiche *pre-emptive*.

4.4 Politiche e meccanismi

A definire la politica di assegnazione della CPU è specializzato lo *scheduler* che, una volta prese le decisioni, affida al *dispatcher* il compito di effettuare il *context switching*.

4.5 Criteri di scheduling

Per analizzare gli algoritmi di *scheduling* si utilizzano principalmente i seguenti indicatori di performance:

- Utilizzo della CPU
- Throughput (del sistema): numero di processi completati nell'unità di tempo
- Tempo di attesa: tempo all'interno della *ready queue*
- Turnaround: tempo tra la sottomissione del *job* ed il suo completamento
- Tempo di risposta: intervallo di tempo tra la sottomissione di un *job* e l'inizio della sua prima risposta (a differenza del *turnaround*, non dipende dai dispositivi di I/O)

In generale, devono essere massimizzati l'utilizzo della CPU ed il *throughput*, minimizzati i restanti.

4.6 Algoritmi di scheduling

4.6.1 FCFS (First-Come-First-Served)

I processi vengono gestiti secondo una logica FIFO non *pre-emptive*.

Problemi Con un algoritmo di questo tipo non è possibile agire sull'ordine dei processi. Un processo che segue un *job* di tipo *CPU bound* è destinato a una lunga attesa.

Effetto conviglio Nel caso in cui molti processi *I/O bound* dovessero seguire un processo *CPU bound*, si avrebbe uno scarso utilizzo della CPU.

4.6.2 SJF (Shortest-job-first)

Per ogni processo nella *ready queue* viene stimata la lunghezza del prossimo CPU *burst* e viene *schedulato* quello con valore minore.

Questo algoritmo presenta anche una versione *pre-emptive* (SRTF Shortest-remaining-time-first) che, all'arrivo di un nuovo processo, ne stima il CPU *burst*. Se questo dovesse essere minore del processo in corso, lo sostituisce.

Problema La stima della lunghezza del prossimo CPU *burst* rappresenta un problema complesso.

Stimare la lunghezza di CPU *burst* Per stimare la lunghezza del prossimo CPU *burst* esistono, per esempio, i seguenti approcci:

- Stimare la lunghezza sulla base del passato
- exponential averagin:

Exponential averaging Ogni termine ha meno peso del termine precedente

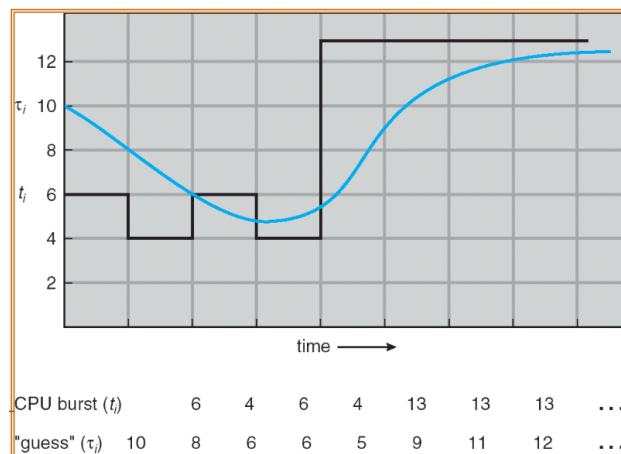
t_n = effettiva lunghezza dell' n^{th} CPU burst

τ_{n+1} = tempo predetto per il prossimo CPU *burst*

$$\alpha, 0 \leq \alpha \leq 1$$

$$\tau_{n+1} = (\alpha t_n) + (1 - \alpha)\tau_n = (\alpha t_n) + (1 - \alpha)(\alpha t_{n-1}) + \dots + (1 - \alpha)^{n+1}\tau_0$$

Se $\alpha = 0$ la storia recente non conta. Se $\alpha = 1$ conta sono l'ultimo valore. Con $\alpha = \frac{1}{2}$:



4.7 Scheduling con priorità

Questo tipo di *scheduling* si basa sul principio di "priorità" di ogni processo. Lo *scheduler* seleziona il processo con la priorità più alta e molteplici processi con lo stesso livello di priorità vengono trattati in maniera FCFS.

4.7.1 Priorità

Ad ogni processo può essere assegnata una priorità in due modi:

- Internamente dal SO sulla base delle sue politiche
- Esternamente dal SO

Un volta assegnata una priorità, questa può essere sia di tipo costante che variabile.

4.7.2 Problema & Soluzione

Il problema tipico di questo tipo di algoritmi è rappresentato dalla *starvation*. I processi con priorità bassa potrebbero rimanere per un tempo indefinito all'interno della *ready queue*.

La soluzione a questo tipo di problema è la modifica dinamica della priorità:

- Feedback negativo: priorità decrescente al crescere del tempo di CPU
- Feedback positivo: priorità cresce dinamicamente con il tempo di attesa del processo

4.7.3 Round Robin

Algoritmo *pre-emptive* che assegna ad ogni processo all'interno della *ready queue* una quantità di tempo CPU prefissata Δt (trattando la coda come FCFS circolare).

L'obiettivo è minimizzare i tempi di risposta trattando tutti i processi in modo egualitario (assenza *starvation*).

Problemi Possono sorgere problemi relativi al Δt (quanto di tempo) e al possibile degrado delle prestazioni del SO dovuto al trattamento egualitario dei processi. In particolare riguardo al quanto di tempo:

- Δt piccolo (ma comunque $\gg t_{\text{context-switching}}$): alta frequenza di *context switch* $\Rightarrow$ *overhead* eccessivo
- Δt grande: *overhead* ridotto, ma tempi di risposta più elevati

4.7.4 Approcci misti

Spesso all'interno di SO vengono utilizzati approcci misti, come il *Multiple Level Feedback Queues*: più code per ogni tipo di processo, ogni coda con priorità diversa e gestita con algoritmi diversi, possibilità di spostare i processi tra le code, ecc.

4.8 Scheduling in UNIX

All'interno dei sistemi UNIX l'obiettivo principale è quello di privilegiare i processi interattivi.

4.8.1 MLFQ

Questo algoritmo presenta circa 160 livelli di priorità. Viene definito un valore detto p_{zero} per cui:

- Priorità $\geq p_{zero}$: processi utente ordinari
- Priorità $< p_{zero}$: processi di sistema

Ad ogni livello è associata una coda di tipo *round robin*, con Δt pari a $100ms$, e per ogni singolo processo viene ricalcolata la priorità ogni secondo seguendo un feedback negativo. È poi possibile modificare manualmente la priorità (in maniera decrescente) attraverso il comando **nice**.

5 Interazione tra processi

Due processi possono essere tra loro interagenti oppure indipendenti. Si dicono del primo tipo se la loro esecuzione è in qualche modo dipendente, e possono ulteriormente essere classificati in due categorie:

- Cooperanti: interagiscono volontariamente per il raggiungimento di un obiettivo comune (generalmente fanno parte della stessa applicazione).
- In competizione: Interagiscono indirettamente per l'acquisizione di risorse comuni.

5.1 Processi interagenti

L'interazione tra due processi può avvenire in due modi:

- Comunicazione: scambio di informazioni
- Sincronizzazione: imposizione di vincoli temporali sull'esecuzione

La realizzazione specifica dipende dal tipo di ambiente nel quale il processo deve essere eseguito. In sistemi ad ambiente locale, per esempio, comunicazione e sincronizzazione possono avvenire unicamente attraverso lo scambio di messaggi ed eventi. In sistemi ad ambiente globale, invece, si usano strumenti di sincronizzazione (lock, semafori, etc.) e variabili condivise.

Tutti i meccanismi di comunicazione e sincronizzazione che non fanno utilizzo di variabili vengono offerti dal sistema operativo.

5.2 Comunicazione

5.2.1 Scambio di messaggi

In riferimento al modello ad ambiente locale, non è presente memoria condivisa. Il SO, quindi, offre dei meccanismi a supporto della comunicazione tra processi (Inter-Process-Communication). Generalmente, le primitive offerte dal sistema sono:

- Send: spedizione di messaggi
- Receive: ricezione di messaggi

5.2.2 Meccanismi di comunicazione

La comunicazione avviene attraverso un canale prestabilito e può avere diverse caratteristiche:

- Tipo di comunicazione: diretta, indiretta, simmetrica, asimmetrica, bufferizzata, etc.
- Caratteristiche del canale: monodirezionale, bidirezionale, capacità, modalità di creazione, uno-a-uno, uno-a-molti, etc.
- Caratteristiche del messaggio: dimensione, tipo.

5.2.3 Naming

La firma delle funzioni di invio può variare a seconda della struttura del sistema di comunicazione:

- Diretta: al messaggio viene associato l'identificatore del processo ricevente

```
send(Proc, msg)
```

- Indiretta: il messaggio viene inviato in una *mailbox*

```
send(Mailbox, msg)
```

5.2.4 Comunicazione diretta

Nella comunicazione diretta, i processi interlocutori devono conoscersi reciprocamente. Per ogni coppia esiste un solo canale di comunicazione, bidirezionale e punto-a-punto.

Simmetrica Entrambi conoscono l'identificativo dell'altro:

```
pid C = /* identificatore consumatore */
int main() {
    msg M;
    do {
        produco(\&M);
        send(C, M);
    } while(0);
}
```

```
pid P = /* identificatore produttore */
int main() {
    msg M;
    do {
        receive(P, \&M);
        consumo(M);
    } while(0);
}
```

Asimmetrica Il consumatore non è tenuto a conoscere a priori l'identificativo del produttore:

```
pid C = /* identificatore consumatore */
int main() {
    msg M;
    do {
        produco(&M);
        send(C, M);
    } while(0);
}
```

```
int main() {
    msg M;
    pid id; /* identificatore che ricaverò */
    do {
        receive(&id, &M);
        consumo(M);
    } while(0);
}
```

5.2.5 Comunicazione indiretta

I processi non sono tenuti a conoscersi reciprocamente. I messaggi vengono inviati all'interno di una *mailbox*, alla quale possono avere accesso diversi processi.

Proprietà Il canale di comunicazione (*mailbox*) non viene generato in automatico, possono connettersi più di due processi e tra due processi possono esserci più canali di comunicazione. Quando una *mailbox* è del tipo multi-a-uno è anche detta "porta".

5.3 Buffering del canale

Ogni canale ha a disposizione una determinata capacità (> 0). In caso di capacità nulla non vi è accodamento perché il canale non è più in grado di contenere informazioni. In una situazione di questo tipo possono essere applicati due approcci:

- Mittente e destinatario devono sincronizzarsi nell'atto di spedire e ricevere (comunicazione sincrona o rendez-vous)

- *send* e *receive* possono essere sospensive

In caso di capacità non nulla, generalmente:

- se la coda non è piena il messaggio viene posto in coda
- se la coda è piena la *send* è sospensiva
- se la coda è vuota la *receive* è sospensiva

5.3.1 Send sincrona/asincrona

A seconda del canale la *send* può essere o non essere sospensiva. Nei canali a capacità nulla la *send* diventa sincrona attendendo lo spazio libero, nei canali a capacità non nulla la *send* diventa asincrona (essendo sempre presente spazio).

5.3.2 Send con sincronizzazioni estesa

Esiste anche un particolare tipo di servizio *send* che permette al mittente di entrare in sospensione fino alla ricezione di una *reply* da parte del destinatario. Il messaggio potrebbe richiedere l'esecuzione di un servizio come il RPC (*Remote Procedure Call*).

5.4 Comunicazione in UNIX

I sistemi di tipo UNIX mettono a disposizione tre meccanismi di comunicazione:

- *pipe*: comunicazione locale (stessa gerarchia di processi)
- *fifo*: comunicazione locale (anche diverse gerarchie)
- *socket*: comunicazione in ambiente distribuito (tra processi in esecuzione su nodi di rete differenti)

5.4.1 pipe

Il *pipe* è un tipo di comunicazione indiretta, unidirezionale, multi-a-molti e bufferizzata. Non possiede un naming esplicito ed ha un comportamento per le *send* e *receive* di tipo sospensivo a canale pieno/vuoto.

5.5 Sincronizzazione

Due processi possono cooperare o competere e, in entrambi i casi, necessitano l'accesso ad una risorsa comune, per la quale sono stabiliti dei vincoli d'accesso.

5.5.1 Mutua esclusione

Con "mutua esclusione" si intende una sincronizzazione per cui due *thread* competono in maniera tale che solamente uno dei due possa avere accesso a una specifica risorsa comune, mentre l'altro attende. Nel momento in cui la risorsa si libera, il *thread* in attesa viene risvegliato.

5.5.2 Sincronizzazione tra processi nel modello ad ambiente globale

Nel modello ad ambiente globale i *thread* possono condividere variabili. In caso di cooperazione possono scambiarsi messaggi attraverso strutture dati condivise e, in caso di competizione, utilizzare tutti quelle stesse variabili. In entrambi i casi nasce la necessità di sincronizzare gli accessi alla memoria condivisa attraverso dei strumenti che devono essere forniti dal SO (*lock*, semafori, etc.).

5.5.3 Sincronizzazione tra processi nel modello ad ambiente locale

Nel modello ad ambiente locale non è possibile condividere memoria. Le risorse condivise vengono unicamente gestite dal SO e la sincronizzazione è unicamente possibile attraverso l'invio di "eventi" tra i processi.

6 Segnali in UNIX

UNIX adotta un modello ad ambiente locale. La sincronizzazione è quindi possibile solamente attraverso l'utilizzo di segnali. Un segnale è un'interruzione software che viene notificata in modo asincrono a un processo, che ne rileva immediatamente la presenza.

Generalmente, un segnale è pensato per notificare un evento e può essere inviato dal *kernel* ai processi oppure da un processo ad altri processi. Quando un processo riceve un *signal* può comportarsi in tre modi:

- Eseguire l'*handler* assegnato a quel segnale.
- Eseguire un'azione predefinita (*default*) dal SO.
- Ignorare il segnale

In entrambi i primi casi, la risposta del processo è di tipo asincrono. Interrompe la sua esecuzione, procede con l'esecuzione dell'azione desiderata e riprende l'esecuzione interrotta.

6.1 Tutti i segnali

Ogni versione di UNIX presenta vari tipi di segnali, ognuno dei quali è identificato da un intero. Ogni segnale è associato a un numero intero e a un handler di default.

Generalmente i segnali seguono la forma **SIGXXX** (SIGKILL, SIGSTOP, etc.) ed è possibile ricavarne l'associazione con il proprio valore intero attraverso l'header `<signal.h>`. Per visualizzare la lista completa di segnali presenti nella propria versione UNIX: `kill -l`.

6.2 Gestione dei segnali

Esistono due segnali per i quali non è possibile stabilire un comportamento differente da quello di default: SIGKILL e SIGSTOP.

Per la gestione dei segnali, sia in ricezione che in invio, il SO fornisce una serie di primitive.

6.2.1 signal

Nome	signal
Descrizione	Assegna uno specifico <i>handler</i> ad un segnale
Parametri	sig – il valore intero del segnale per cui si vuole definire l' <i>handler</i> (<i>*func</i>)(int) – nuovo <i>handler</i> , SIG_IGN per ignorare il segnale, SIG_DFL per l'azione di default
Valore di ritorno	Puntatore al precedente <i>handler</i> , SIG_ERR in caso di errore
Libreria	signal.h
Firma	<pre>#include <signal.h> void (*signal(int sig, void (*func)(int)));</pre>

6.2.2 kill

Nome	kill
Descrizione	Invia uno specifico segnale a un processo o un gruppo di processi
Parametri	<code>pid</code> – <i>pid</i> del destinatario, 0 per tutto il gruppo, <-1 per tutti i processi con <code>groupId</code> pari a <code> pid </code> <code>sig</code> – segnale da inviare
Valore di ritorno	0 – successo 1 – fallimento 64 - successo parziale
Libreria	signal.h
Firma	<pre>#include <signal.h> int kill(pid_t pid, int sig);</pre>

6.2.3 sleep

Nome	sleep
Descrizione	Sospensione del processo per un numero di secondi. Se il processo riceve un segnale, si risveglia prematuramente.
Parametri	<code>n</code> – numero di secondi
Valore di ritorno	0 – sospensione non interrotta <code>k</code> – numero di secondi rimanenti alla sleep interrotta
Libreria	signal.h
Firma	<pre>#include <signal.h> unsigned int sleep(unsigned int n);</pre>

6.2.4 alarm

Nome	alarm
Descrizione	Invia al processo il segnale <code>SIGALRM</code> dopo un numero di secondi.
Parametri	<code>n</code> – numero di secondi
Valore di ritorno	0 – nessun time-out impostato in precedenza <code>k</code> – numero di secondi rimanenti al precedente time-out
Libreria	signal.h
Firma	<pre>#include <signal.h> unsigned int alarm(unsigned int n);</pre>

6.2.5 pause

Nome	pause
Descrizione	Sospende il processo fino alla ricezione di qualunque segnale.
Parametri	void
Valore di ritorno	-1
Libreria	signal.h
Firma	<pre>#include <unistd.h> int pause(void);</pre>

6.3 Segnali e multiprogrammazione

Le associazioni tra segnale ed *handler* vengono registrate all'interno della *user structure*.

6.3.1 fork

Copiando la *user structure* del processo generatore, la **fork** permette ai figli di ereditare le informazioni relative ai segnali del padre. Successive **signal**, però, modificano solo la *user structure* del figlio, non intaccando il padre.

6.3.2 exec

A seguito di una **exec**, la *user structure* di un processo viene clonata a meno delle informazioni legate al codice. Questo implica che, a seguito di una **exec**, il processo ignorerà e gestirà in modo default gli stessi segnali del padre, perdendo però gli *handler* personalizzati.

6.4 Handlers

Gli *handler* sono funzioni eseguite alla ricezione di un segnale. Richiedono sempre un parametro in ingresso, che rappresenta il segnale ricevuto, e non restituiscono mai alcun valore.

L'associazione segnale/*handler* non è sempre durevole. In alcune versioni di UNIX, una volta eseguito l'*handler*, questo viene riportato a quello predefinito.

6.4.1 Affidabilità dei segnali

È compito di un SO definire una politica che gestisca il comportamento dei processi alla ricezione di un segnale durante l'esecuzione di un *handler*. Esistono tre tecniche:

- Innestamento delle routine
- Perdita del segnale
- Accodamento dei segnali

6.4.2 Interrompibilità delle system call

Anche le *system call* sono soggette a interruzione da parte dei segnali. Se ne definiscono due gruppi:

- *slow system call*: possono richiedere un'onerosa quantità di tempo, anche dovuta ad attese
- *system call* non-slow

Una *slow system call* è interrompibile dai segnali (ritornando -1 e impostando *errno* a *EINTR*) e può seguire una delle seguenti politiche:

- Riesecuzione automatica della *system call*
- Esecuzione non automatica ma comandata dal processo

6.5 Segnali in Linux

All'interno dei sistemi Linux gli *handler* sono persistenti e, alla ricezione di un nuovo segnale, questo viene gestito una volta terminato l'*handler* corrente. Se durante l'esecuzione dell'*handler* per un certo segnale viene ricevuto lo stesso segnale, questo viene "accorpato" al precedente.

7 File system

Con *file system* si indica quella parte del SO che permette la gestione delle informazioni sulla memoria di massa.

7.1 Organizzazione del file system

La struttura di un *file system* può essere rappresentata da un insieme di componenti di vari livelli:

1. Struttura logica: presenta una visione astratta basata su file, direttori e partizioni. Rende anche disponibili una serie di operazioni sulla visione proposta: copia, cancellazione, spostamento, etc.
2. Accesso: definisce e realizza i meccanismi per accedere al contenuto dei file, in particolare:
 - Definisce l'unità di trasferimento da/verso file (record logico)
 - Realizza i metodi d'accesso (sequenziale, casuale, ad indice)
 - Realizza i meccanismi di protezione
3. Organizzazione fisica: rappresentazione di file e direttori su dispositivo:
 - Allocazione dei file sul dispositivo: mapping di record logici su blocchi
 - Rappresentazione della struttura logica
4. Dispositivo Virtuale: vista astratta del dispositivo come sequenza di blocchi di uguale grandezza

7.2 Struttura logica

7.2.1 File

Un *file* è un insieme di informazioni. Ogni *file* è individuato da almeno un nome simbolico ed è caratterizzato da un insieme di attributi.

Attributi Gli attributi di un file possono variare a seconda del SO. Generalmente:

- Tipo
- Indirizzo: puntatore a memoria secondaria
- Dimensione
- Data e ora (della creazione)

In sistemi multiutente sono solitamente presenti anche il proprietario e la protezione applicata.

Descrittore Rappresenta la struttura dati che contiene gli attributi del *file*. Ogni descrittore deve essere mantenuto dal SO in maniera permanente.

7.2.2 Directory

È una struttura utilizzata per organizzare i *file* all'interno del *file system*.

Operazioni Le operazioni possibili su una *directory* sono:

- Creazione e distruzione
- Aggiunta e cancellazione
- Listing (elenco del contenuto)
- Attraversamento
- Ricerca

Tipi La struttura di una *directory* può variare sulla base del SO. Gli schemi più comuni sono:

- A un livello: all'interno del *file system* è presente solamente una *directory*.
- A due livelli: il primo livello è rappresentato dalle *directory* dedicate ai singoli utenti del sistema mentre il secondo alle *directory* per i dati di un singolo utente.
- Struttura ad albero: organizzazione a N livelli. Ogni *directory* può contenere sia *file* che altre *directory*.
- Struttura a grafo aciclico: estensione della struttura ad albero con l'aggiunta di *link* differenti allo stesso *file*.

7.3 Mounting

Per poter utilizzare un nuovo disco all'interno del sistema, la maggior parte dei SO richiede *mounting* esplicito, con il quale si rende disponibile una nuova porzione di memoria.

7.4 Operazioni sui file

È responsabilità del SO garantire un accesso *on-line* efficiente e coerente ai *file*. Quando un processo modifica un *file*, è fondamentale che tali modifiche siano immediatamente visibili anche agli altri processi che accedono allo stesso *file*.

Il sistema operativo fornisce un insieme di operazioni fondamentali per la gestione dei file:

- **Creazione:** allocazione dello spazio su memoria secondaria e definizione degli attributi del file.
- **Lettura:** accesso ai record logici contenuti nel file.
- **Scrittura:** inserimento o modifica dei record logici.
- **Cancellazione:** rimozione del file dal *file system*.

Per ottimizzare l'accesso ai *file* e ridurre i tempi di ricerca, il SO mantiene una tabella dei *file* attualmente aperti. Inoltre, implementa il meccanismo di *memory mapping*, che consente di mappare un *file* (o una sua porzione) direttamente nello spazio di indirizzamento della memoria centrale, migliorando così le prestazioni di lettura e scrittura.

All'interno della tabella dei file aperti, il SO gestisce due operazioni essenziali:

- **Apertura:** inserimento del *file* nella tabella e, se necessario, attivazione del *memory mapping*.
- **Chiusura:** salvataggio delle modifiche sulla memoria secondaria e rimozione del *file* dalla tabella.

7.5 Struttura interna del file

Ogni *file*, dal punto di vista dell'accesso, viene visualizzato dai processi come un insieme di "record logici". Dal punto di vista del dispositivo virtuale, invece, i *file* vengono memorizzati come insieme di blocchi (o record fisici).

Uno dei compiti di un SO è stabilire l'associazione tra record logici e fisici. Solitamente le dimensioni del blocco sono maggiori di quelle del record logico.

7.5.1 Blocchi

Si definisce blocco un'unità di trasferimento fisico nelle operazioni di I/O, a dimensione sempre fissa.

7.5.2 Record logici

Si definisce record logico un'unità di trasferimento logico nelle operazioni di accesso al file.

7.6 Metodi d'accesso

È possibile utilizzare diversi metodi per accedere ad un *file*, ognuno dei quali è indipendente dal tipo di dispositivo utilizzato e dalla tecnica di allocazione dei blocchi.

7.6.1 Sequenziale

Il *file* è una sequenza di record logici. Per accedere ad un particolare record è necessario accedere ai $i-1$ record precedenti. Sono disponibili operazioni del tipo **readnext** e **writenext**, che si basano sulla registrazione della posizione successiva al record corrente (puntatore al *file*).

7.6.2 Diretto

Il *file* è un insieme di record logici numerati. Vengono rese disponibili funzioni del tipo **read-i** e **write-i**, solitamente utili per l'accesso a file di grosse dimensioni.

7.6.3 A indice

Ai *file* viene associata una struttura dati che permette l'accesso alle informazioni attraverso una ricerca sull'indice, successivamente associato al record logico.

7.7 Protezione

È compito dei SO fornire la possibilità ai creatori/proprietari dei *file* di controllare l'accesso ad una serie di comandi sul *file* da parte di un utente o di un gruppo di utenti.

In UNIX, vengono definite tre modalità d'accesso:

- read
- write
- execute

su tre classi di utenti:

- user
- group
- others

Ad un amministratore è permesso creare gruppi con nomi unici e modificarne gli utenti.

Dato un *file* o una *directory* è importante definirne i permessi.

7.8 Organizzazione fisica

Il SO si occupa dell'implementazione del *file system* all'interno dei dispositivi di memorizzazione secondaria. In particolare: realizzazione dei descrittori, allocazione dei blocchi fisici, gestione dello spazio libero, ecc.

7.9 Metodi di allocazione

I metodi d'allocazione rappresentano la tecnica utilizzata dal SO per immagazzinare i blocchi di un *file* all'interno della memoria secondaria. Alcuni SO possono utilizzare più tecniche, a seconda della dimensione del file.

7.9.1 Allocazione contigua

I *file* sono mappati su un insieme di blocchi fisicamente contigui.

Vantaggi I *file* hanno un basso costo di ricerca e accesso.

Svantaggi Può essere dispendioso individuare spazi liberi. Potrebbero, poi, presentarsi frammentazioni esterne (blocchi liberi contigui sempre più piccoli) che necessitano azioni di compattazione.

7.9.2 Allocazione a lista concatenata

I blocchi di un *file* vengono organizzati mediante una lista concatenata.

Vantaggi Minor corso di allocazione.

Svantaggi I costi di ricerca crescono, così come lo spazio occupato (necessità di immagazzinare il link ai blocchi successivi). Possibile perdita di dati per danneggiamento dei link.

7.9.3 Tabella di allocazione dei file (FAT)

Estensione della struttura a lista concatenata. Per ogni partizione viene mantenuta una tabella (*FAT*) con una riga per ogni blocco. Il concatenamento di blocchi dei *file* è salvato all'interno della tabella.

7.9.4 Allocazione ad indice

Tutti i puntatori ai blocchi utilizzati da un *file* sono salvati all'interno di un unico blocco detto: blocco indice.

Vantaggi Stessi dell'allocazione a lista più la possibilità d'accesso diretto e una maggiore velocità d'accesso

Svantaggi Potrebbe esserci uno scarso utilizzo dei blocchi ad indice

8 File system in UNIX

All'interno del *file system* UNIX tutto è un *file*, suddiviso in tre categorie:

- ordinari
- *directory*
- speciali (rappresentano dispositivi fisici nella *directory* */dev*)

Ad ogni *file* possono essere associati uno o più nomi simbolici, ma ad un solo descrittore (*i-node*) con il relativo identificatore numerico (*i-number*).

8.1 Organizzazione fisica

I sistemi UNIX utilizzano un'allocazione ad indice con dischi formattati in blocchi (512 – 4096 bytes) e divisi in settori:

1. boot block (procedure di inizializzazione del sistema)
2. super block (limiti delle regioni, puntatore a lista di blocchi liberi, puntatore a lista degli *i-node* liberi)
3. i-list (lista di tutti gli *i-node* dei file)
4. data blocks (memorizzazione dei *file*)

8.1.1 i-node

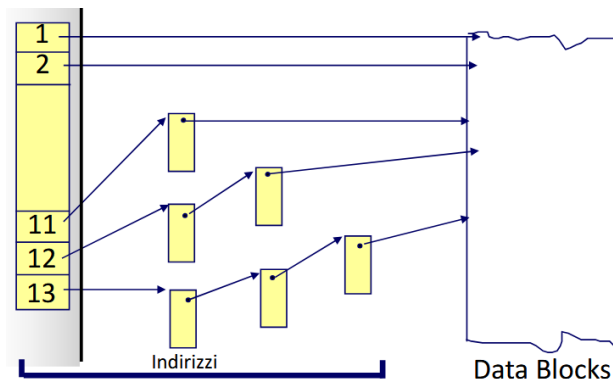
Con *i-node* si intendono i descrittori dei *file*. Gli attributi legati ad ogni *i-node* sono:

- tipo del file
- proprietario, gruppo
- dimensione
- data
- 12 bit di protezione
- numero di links
- 13-15 indirizzi di blocchi

8.1.2 Indirizzamento

L'allocazione dei *file* in UNIX non è su blocchi contigui. All'interno degli *i-node*, quindi, è necessario immagazzinare l'indirizzo dei blocchi contenenti l'effettivo indirizzo dei dati all'interno della memoria secondaria. In particolare, ogni *i-node* contiene 10 indirizzi, ripartiti nei seguenti modi:

- | | |
|--|----------------|
| 1 Blocchi di dati (indirizzamento diretto) | (blocchi 1-10) |
| 2 Indirizzo di un blocco contenente indirizzi ad altri blocchi | (blocco 11) |
| 3 Indirizzi a due livelli di indirettezza | (blocco 12) |
| 4 Indirizzi a tre livelli di indirettezza | (blocco 13) |



Supponendo un blocco di dimensione 512 bytes e indirizzi a 32 bit, ogni blocco può contenere 128 indirizzi. Quindi:

- Con indirizzazione diretta: $10 \times 512 \text{ bytes} = 5 \text{ KiB}$
- Con indirizzazione singola: $128 \times 512 \text{ bytes} = 64 \text{ KiB}$
- Con indirizzazione doppia: $128 \times 128 \times 512 \text{ bytes} = 8 \text{ MiB}$
- Con indirizzazione tripla: $128 \times 128 \times 128 \times 512 \text{ bytes} = 1 \text{ GiB}$

Di conseguenza, un sistema con blocchi di queste dimensioni non può indirizzare un file di dimensioni maggiori di $1 \text{ GiB} + 8 \text{ MiB} + 64 \text{ KiB} + 5 \text{ KiB}$.

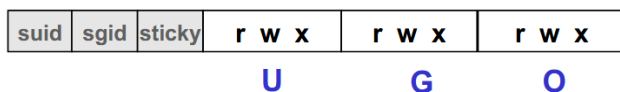
8.2 Protezione

È possibile accedere ad un file in tre modi: lettura, scrittura ed esecuzione. I sistemi UNIX permettono al proprietario del file di concedere o negare alcuni di questi accessi ad alcuni utenti e/o gruppi. Esiste un utente privilegiato che ha accesso a qualunque file (**root**).

8.2.1 Bit di protezione

Ad ogni *file* vengono associati 12 bit di protezione:

- | | | |
|---|-------------------------------------|-------------|
| 1 | <i>Set-User-ID</i> | (bit 1) |
| 2 | <i>Set-Group-ID</i> | (bit 2) |
| 3 | <i>Save-Text-Image</i> | (bit 3) |
| 4 | Permessi utente proprietario | (bit 4-6) |
| 5 | Permessi gruppo utente proprietario | (bit 7-9) |
| 6 | Permessi altri utenti | (bit 10-12) |



I primi tre bit di protezione si riferiscono, e quindi sono significativi, a *file* eseguibili. Quando eseguito, un processo è associato dinamicamente allo *user-id* e al *group-id* dell'utente che l'ha eseguito. Quando *Set-User-ID* vale uno, al processo eseguito viene associato lo *user-id* del proprietario del file. Segue lo stesso ragionamento *Set-Group-ID*. Attraverso questi bit, chi esegue il processo può assumere temporaneamente l'identità del proprietario. Con *Save-Text-Image*, invece, si indica il desiderio di mantenere in area di *swap* il processo, anche dopo la sua terminazione.

Un esempio di utilizzo dei bit **SUID** e **SGID**, entrambi impostati a 1, è rappresentato dal programma `/usr/bin/passwd`, di proprietà dell'utente **root**. Questo deve poter accedere e modificare il file `/etc/shadow`, anche quando viene eseguito da utenti non privilegiati. Poiché `/etc/shadow` è di proprietà di **root** e ha permessi di scrittura riservati esclusivamente a tale utente, il bit **SUID** consente al programma di essere eseguito con i privilegi dell'utente proprietario (cioè **root**), indipendentemente da chi lo esegua.

Aggiornamento è possibile modificare i permessi di un file attraverso il comando e la system call **chmod**.

8.3 Directory

Anche le *directory* sono memorizzate come file all'interno del *file system* di UNIX. In particolare, ciascuna directory è rappresentata come una tabella che contiene due colonne: il nome del file e il relativo *i-number*. Per ogni file presente nella directory, è presente una riga nella tabella che associa il nome del file al suo *i-number*.

8.4 Implementazioni del file system in UNIX

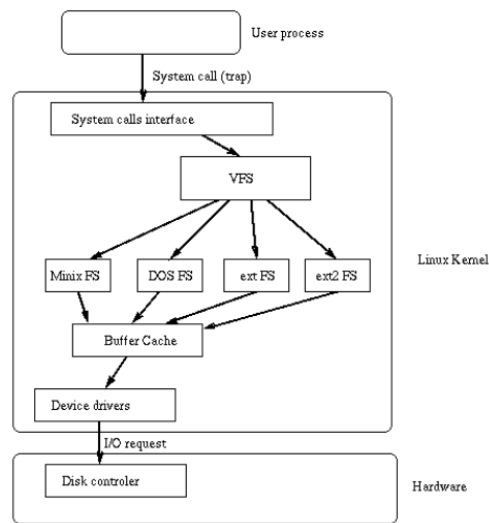
Minix Limitazioni su indirizzabilità (64MiB per file), nomi dei file (16 caratteri) e gestione dei blocchi.

Ext FS (1992) Indirizzabilità a 1 GiB/4 GiB e gestione blocchi liberi con liste.

Ext 2 (1993) Indirizzabilità a 16 GiB/4 TiB. Viene concessa all'amministratore la possibilità di modificare le dimensioni dei blocchi (1024-4096 bytes) e delle *i-list*. La memoria secondaria viene divisa in gruppi, ognuno dei quali contenente *data blocks*, *i-node* e una copia di *superblock* e descrittori del *file system*.

Ext 3 (2001), Ext 4 Estensioni di Ext 2, con maggiore tolleranza ai guasti.

Virtual File System Linux supporta l'integrazione di diversi tipi di *file system*, anche differenti da Ext. Questo è possibile grazie al *Virtual File System* (VFS), un livello di astrazione che intercetta tutte le *system call* relative alle operazioni sui file e le adatta in base al tipo di *file system* utilizzato. Il VFS fornisce un'interfaccia unificata, mappando ogni *file system* su oggetti interni al *kernel*, ciascuno con le proprie specifiche e strutture dati, garantendo così compatibilità tra file system eterogenei.



8.5 Accesso a file

L'accesso ai file è di tipo sequenziale e non presenta alcun tipo di strutturazione. È presente un puntatore al file, *I/O Pointer*, che registra la posizione corrente per tutte le operazioni di accesso (lettura, scrittura e lettura/scrittura).

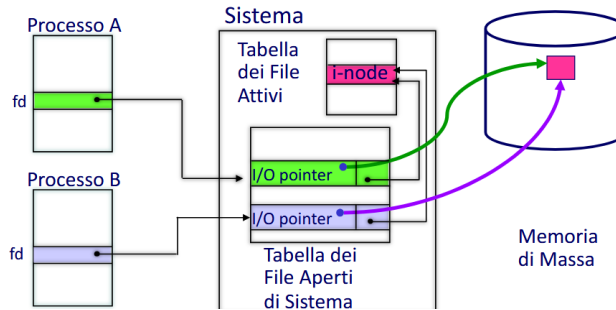
8.5.1 File descriptor

A ogni processo è associata una tabella dei file aperti, di dimensione generalmente limitata, allocata all'interno della *user structure*. Ogni voce della tabella corrisponde a un file attualmente aperto e viene identificata da un indice intero, chiamato *file descriptor*. I file descriptor 0, 1 e 2 sono riservati e corrispondono rispettivamente allo *standard input*, *standard output* e *standard error*.

8.5.2 Strutture dati e kernel

Il SO utilizza due strutture dati globali per realizzare l'accesso ai file, allocate nell'area dati del *kernel*.

1. Tabella dei file attivi: per ogni file aperto contiene una copia del suo *i-node* (in maniera tale da evitare accessi al disco per ottenere gli attributi dei file acceduti).
2. Tabella dei file aperti di sistema: per ogni operazione di apertura viene inserito un elemento contenente *I/O Pointer* e puntatore all'*i-node* nella tabella dei file attivi. Può essere quindi presente più volte lo stesso file, se aperto da processi differenti, con *I/O Pointer* distinti, in maniera tale da permettere l'accesso contemporaneo.



Condivisione tra padre e figlio Quando un padre apre un file e poi crea un figlio, quest'ultimo eredita la stessa *User Structure*. Il figlio potrà, quindi, accedere al file senza doverlo aprire ed ogni modifica dell'*I/O Pointer* è condivisa tra entrambi.

8.6 System call per l'accesso ai file

8.6.1 Open

Nome	open
Descrizione	Aprire un file
Parametri	pathname – nome del file flag – metodo di accesso (O_RDONLY, O_WRONLY, O_APPEND, O_CREAT, O_TRUNC) mode – bit di protezione nel caso di creazione del file
Valore di ritorno	fd – <i>file descriptor</i> associato al file -1 – fallimento
Libreria	sys/stat.h
Firma	<pre>#include <fcntl.h> int open(const char *pathname, int flag, int mode);</pre>

Conseguenze Se il file viene aperto con successo, l'*I/O Pointer* viene posizionato all'inizio del file (se non viene specificato il flag O_APPEND).

8.6.2 Creat

Nome	creat
Descrizione	Crea un nuovo file
Parametri	pathname – nome del file mode – protezione del file
Valore di ritorno	fd – <i>file descriptor</i> del file creato -1 – fallimento
Libreria	sys/stat.h
Firma	<pre>#include <fcntl.h> int creat(char nomefile[], int mode);</pre>

Conseguenze Se il file viene creato con successo, l'*I/O Pointer* viene posizionato all'inizio del file.

8.6.3 Close

Nome	close
Descrizione	Chiude un file aperto
Parametri	fd – <i>file descriptor</i> del file da chiudere
Valore di ritorno	0 – successo <0 – fallimento
Libreria	sys/stat.h
Firma	<pre>#include <fcntl.h> int close(int fd);</pre>

Conseguenze Se `close` ha successo, il file viene memorizzato su disco, eliminato l'indice `fd` dalla tabella dei file aperti ed eliminati gli elementi corrispondenti all'interno delle tabelle dei file aperti e attivi di sistema

8.6.4 Read

Nome	read
Descrizione	Legge un numero di caratteri n , in maniera atomica, a partire dalla posizione dell' <i>I/O Pointer</i> e li inserisce nel buffer.
Parametri	<code>fd</code> – descrittore del file <code>*buf</code> – buffer di lettura <code>n</code> – numero di caratteri
Valore di ritorno	<code>k</code> – caratteri effettivamente letti <code><0</code> – caratteri effettivamente letti
Libreria	<code>fcntl.h</code>
Firma	<pre>#include <fcntl.h> int read(int fd, char *buf, int n);</pre>

Conseguenze L'esecuzione con successo di una `read` sposta l'*I/O Pointer* di k posizioni in avanti, equivalenti al numero di byte effettivamente letti.

8.6.5 Write

Nome	write
Descrizione	Scrive un numero di caratteri n dal buffer a partire dalla posizione dell' <i>I/O Pointer</i> .
Parametri	<code>fd</code> – descrittore del file <code>*buf</code> – buffer di lettura <code>n</code> – numero di caratteri
Valore di ritorno	<code>k</code> – caratteri effettivamente scritti <code><0</code> – fallimento
Libreria	<code>fcntl.h</code>
Firma	<pre>#include <fcntl.h> int write(int fd, char *buf, int n);</pre>

Conseguenze Così come per la `read`, l'operazione `write` (in caso di successo) sposta l'*I/O Pointer* di n posizioni.

8.6.6 Lseek

Nome	lseek
Descrizione	Sposta l' <i>I/O Pointer</i> di un file
Parametri	<code>fd</code> – descrittore del file <code>offset</code> – spostamento rispetto all'origine (byte) <code>origin</code> – origine dello spostamento. Può valere <code>SEEK_SET</code> (inizio), <code>SEEK_CUR</code> (posizione corrente) o <code>SEEK_END</code> (fine),
Valore di ritorno	<code>k</code> – nuova posizione <0 – fallimento
Libreria	fcntl.h
Firma	<pre>#include <fcntl.h> int lseek(int fd, int offset, int origine);</pre>

8.7 System call per la gestione dei file

8.7.1 Unlink

Nome	unlink
Descrizione	Decrementa di uno il numero di link del file.
Parametri	<code>pathname</code> – path del file
Valore di ritorno	0 – successo -1 – fallimento
Libreria	unistd.h
Firma	<pre>#include <unistd.h> int unlink(const char *pathname);</pre>

Conseguenze Se, una volta decrementato, il numero di link è pari a 0 il file viene eliminato.

8.7.2 Link

Nome	link
Descrizione	Aggiunge un link ad un file esistente
Parametri	<code>oldname</code> – nome del file esistente <code>newname</code> – nome associato al nuovo link
Valore di ritorno	0 – successo -1 – fallimento
Libreria	unistd.h
Firma	<pre>#include <unistd.h> int link(const char *oldname, const char * newname);</pre>

Conseguenze Oltre ad aggiornare il numero di link associato al file, la funzione aggiorna il direttorio inserendo un nuovo elemento.

Problemi La funzione potrebbe fallire se `oldname` non esiste, `newname` esiste già oppure `oldname` e `newname` appartengono a *file system* differenti.

8.7.3 Access

Nome	access
Descrizione	Verifica i diritti di un utente su un file
Parametri	<code>pathname</code> – nome del file <code>amode</code> – diritto da verificare
Valore di ritorno	0 – permesso posseduto -1 – permesso non posseduto o fallimento
Libreria	sys/stat.h
Firma	<pre>#include <sys/stat.h> int access(const char *pathname, int amode);</pre>

Conseguenze La funzione restituisce un risultato rispetto al reale `User-ID` del processo (e non l'effettivo).

8.7.4 Stat

Nome	stat
Descrizione	Restituisce gli attributi di un file
Parametri	pathname – nome del file buf – attributi del file (output)
Valore di ritorno	0 – successo -1 – fallimento
Libreria	sys/stat.h
Firma	<pre>#include <sys/stat.h> int stat(const char *pathname, struct stat * buf);</pre>

Conseguenze La funzione restituisce una struttura di questo tipo:

```
struct stat {
    dev_t st_dev; /* ID of device containing file */
    ino_t st_ino; /* i-number */
    mode_t st_mode; /* protection & file type */
    nlink_t st_nlink; /* number of hard links */
    uid_t st_uid; /* user ID of owner */
    gid_t st_gid; /* group ID of owner */
    dev_t st_rdev; /* device ID (if special file) */
    off_t st_size; /* total size, in bytes */
    blksize_t st_blksize; /* blocksize for file system I/O */
    blkcnt_t st_blocks; /* number of blocks allocated */
    time_t st_atime; /* time of last access */
    time_t st_mtime; /* time of last modification */
    time_t st_ctime; /* time of last status change */
};
```

Dalla libreria sys/stat.h vengono fornite alcune macro per verificare le informazioni più comuni:

```
S_ISREG(mode) /* Il file e' regolare? */
S_ISDIR(mode) /* Il file e' una directory? */
S_ISCHR(mode) /* Il file e' di tipo caratteri (speciale)? */
S_ISBLK(mode) /* Il file e' un dispositivo a blocchi (speciale)? */
/* etc */
```

8.7.5 Chmod

Nome	chmod
Descrizione	Modifica i bit di protezione di un file
Parametri	<code>pathname</code> – nome del file <code>mode</code> – nuova protezione del file
Valore di ritorno	0 – successo -1 – fallimento
Libreria	sys/stat.h
Firma	<pre>#include <sys/stat.h> int chmod(const char *pathname, mode_t mode);</pre>

8.7.6 Chown

Nome	chown
Descrizione	Modifica il proprietario ed il gruppo di un file
Parametri	<code>pathname</code> – nome del file <code>owner</code> – id del nuovo proprietario <code>group</code> – id del nuovo gruppo
Valore di ritorno	0 – successo -1 – fallimento
Libreria	sys/stat.h
Firma	<pre>#include <sys/stat.h> int chown(const char *pathname, int owner, int group);</pre>

Protezione La funzione `chown` può essere eseguita esclusivamente da `root`.

8.8 System call per la gestione dei direttori

Alcune delle funzioni per la gestione dei direttori utilizzano tipi astratti perchè dipendenti dall'implementazione.

8.8.1 Chdir

Nome	chdir
Descrizione	Cambia il direttorio corrente
Parametri	nomedir – nome del direttorio
Valore di ritorno	0 – successo -1 – fallimento
Libreria	unistd.h
Firma	<pre>#include <unistd.h> int chdir(const char *nomedir);</pre>

8.8.2 Opendir

Nome	opendir
Descrizione	Apri un direttorio
Parametri	nomedir – nome del direttorio
Valore di ritorno	dir – puntatore alla struttura rappresentante il direttorio aperto NULL – fallimento
Libreria	dirent.h
Firma	<pre>#include <dirent.h> DIR *opendir(const char *nomedir);</pre>

8.8.3 Closedir

Nome	closedir
Descrizione	Chiude un direttorio
Parametri	dir – struttura associata al direttorio da chiudere
Valore di ritorno	0 – successo -1 – fallimento
Libreria	dirent.h
Firma	<pre>#include <dirent.h> DIR *opendir(const char *nomedir);</pre>

8.8.4 Readdir

Nome	readdir
Descrizione	Legge un elemento del direttorio
Parametri	dir – direttorio
Valore di ritorno	dirent – elemento letto NULL – fallimento
Libreria	dirent.h
Firma	<pre>#include <dirent.h> DIRENT *readdir(const char *nomedir);</pre>

Conseguenze Un'operazione di `readdir` restituisce un puntatore alla struttura `dirent` che rappresenta un elemento del direttorio.

La struttura `dirent` è strutturata nel seguente modo:

```
struct dirent {
    long d_ino; /* i-number */
    off_t d_off; /* offset del prossimo */
    unsigned short d_reclen; /* lunghezza del record */
    unsigned short d_namelen; /* lunghezza del nome */
    char *d_name; /* nome del file */
}
```

8.8.5 Mkdir

Nome	mkdir
Descrizione	Crea un direttorio
Parametri	pathname – nome del direttorio mode – bit di protezione
Valore di ritorno	0 – successo <0 – fallimento
Libreria	dirent.h
Firma	<pre>#include <dirent.h> int mkdir(const char *pathname, int mode);</pre>

8.9 Comunicazione tra i processi in UNIX

In ambiente UNIX (e Linux), i processi non possono condividere direttamente la memoria. È quindi necessario ricorrere a meccanismi che consentano la condivisione di dati tra processi distinti. Tali meccanismi si possono suddividere in due categorie principali:

- Condivisione di file
- Utilizzo di strumenti di *Inter Process Communication* (IPC), come *pipe*, *FIFO* e *socket*

I meccanismi di *Inter Process Communication* si distinguono ulteriormente in base al contesto in cui avviene la comunicazione:

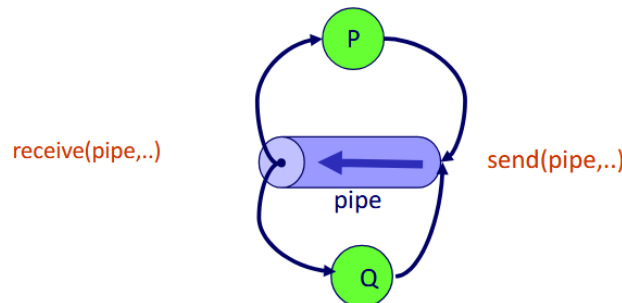
- Comunicazione tra processi sulla stessa macchina
- Comunicazione tra processi su macchine diverse tramite rete

8.9.1 Pipe

La *pipe* è una struttura di tipo unidirezionale, multi-a-molti, a capacità limitata e accessibile con le stesse *system call* dedicate ai file. La gestione dell'ordinamento di consegna dei messaggi è di tipo *fifo*.

Comunicazione La comunicazione tra processi tramite *pipe* è di tipo indiretto: i processi non specificano esplicitamente il destinatario del messaggio, ma si limitano a scrivere o leggere da un canale di comunicazione condiviso, simile a una *mailbox*. Affinché ciò sia possibile, i processi devono condividere la stessa *pipe*, il che richiede che appartengano alla stessa gerarchia di processo, ovvero che abbiano un antenato comune (tipicamente il processo padre).

Un processo può utilizzare una *pipe* sia per l'invio che per la ricezione di dati. Se l'accesso è gestito in modo rigoroso e coordinato, è possibile realizzare una comunicazione bidirezionale, sebbene le *pipe* tradizionali siano progettate per un flusso unidirezionale.



Sincronizzazione Una *pipe*, per sua natura, ha una dimensione limitata e implementa un meccanismo di sincronizzazione automatica tra i processi comunicanti. Le *system call* utilizzate per la lettura e la scrittura su una *pipe* presentano un comportamento di tipo sospensivo, ovvero bloccano l'esecuzione del processo chiamante in determinate condizioni:

- **Sospensione in scrittura:** si verifica quando la *pipe* è piena; il processo scrivente viene sospeso fino a che non si libera spazio.
- **Sospensione in lettura:** si verifica quando la *pipe* è vuota; il processo lettore viene sospeso fino a che non sono disponibili dati da leggere.

Chiusura Ogni processo può chiudere uno o entrambi gli estremi di una *pipe*. Tuttavia, un'estremità della *pipe* è considerata definitivamente chiusa solo quando tutti i processi che la utilizzano l'hanno chiusa.

Se un processo tenta di leggere da una *pipe* priva di scrittori attivi (cioè con l'estremità di scrittura chiusa da tutti), viene restituito il valore 0, indicando la fine del flusso. Al contrario, se un processo tenta di scrivere su una *pipe* che non ha lettori attivi, viene restituito -1 e generato un segnale **SIGPIPE**.

Svantaggi Le *pipe* presentano alcune limitazioni significative:

- **Ambito ristretto:** la comunicazione è possibile solo tra processi che appartengono alla stessa gerarchia, ovvero con un antenato comune.
- **Non persistenza:** una *pipe* esiste esclusivamente durante la vita dei processi che la utilizzano; viene automaticamente distrutta alla terminazione di tutti i processi coinvolti.

8.9.2 FIFO

Una *FIFO* (First In, First Out), anche detta *named pipe*, è una variante della *pipe* tradizionale che possiede un nome nel *file system*, rendendola così accessibile anche a processi che non appartengono alla stessa gerarchia. A differenza delle *pipe* anonime, una *FIFO* è persistente: esiste come oggetto nel *file system* finché non viene eliminata esplicitamente. Inoltre, presenta attributi tipici dei file regolari, come:

- Un proprietario
- Un insieme di permessi di accesso
- Una dimensione massima del buffer

8.10 System call per la comunicazione

8.10.1 pipe

Nome	pipe
Descrizione	Crea una pipe
Parametri	<code>fd[2]</code> – <i>file descriptor</i> per i lati che saranno, rispettivamente, di lettura e di scrittura
Valore di ritorno	0 – successo <0 – fallimento
Libreria	<code>unistd.h</code>
Firma	<pre>#include <unistd.h> int pipe(int fd[2]);</pre>

Conseguenze Se la *system call* ha esito positivo, vengono aggiunti due nuovi elementi alla tabella dei file aperti del processo e ne vengono assegnati i corrispettivi *file descriptor* agli elementi del vettore passato per argomento. In particolare, il primo elemento rappresenta il *file descriptor* del primo elemento aggiunto alla tabella, associato all'estremità di lettura della *pipe*, mentre il secondo al *file descriptor* del secondo elemento, associato all'estremità di scrittura.

Utilizzo Una volta creata una *pipe*, i processi possono utilizzarla usando le stesse *system call* associate alla lettura e alla scrittura su file (`read` e `write`).

8.10.2 mkfifo

Nome	mkfifo
Descrizione	Crea una pipe fifo
Parametri	<code>pathname</code> – nome della <i>fifo</i> <code>mode</code> – permessi sul file
Valore di ritorno	0 – successo <0 – fallimento
Libreria	sys/stat.h
Firma	<pre>#include <sys/stat.h> int mkfifo(const char* pathname, int mode);</pre>

Utilizzo Una volta creata, una *fifo* può essere utilizzata con le stesse *system call* dedicate ai file (`open`, `close`, `read` e `write`).

8.11 Ridirezione di I/O e Piping

La shell di Linux mette a disposizione degli operatori per la ridirezione (ovvero lo spostamento) ed il piping (ovvero l'esecuzione in serie) di comandi.

8.11.1 Ridirezione in output

\$ Comando > file: scrive nel file specificato il risultato dell'esecuzione del comando.

\$ Comando >> file: aggiunge alla fine del file il risultato dell'esecuzione del comando.

Implementazione Per effettuare la ridirezione dell'output, la *shell* crea un processo figlio che esegue i seguenti passaggi:

1. Chiude il *file descriptor* 1 (associato allo `stdout`).
2. Apre il file di destinazione dell'output, che assumerà il *file descriptor* 1.
3. Esegue il comando richiesto tramite la *system call* `exec`.

In questo modo, ogni operazione di scrittura su `stdout` da parte del comando verrà automaticamente redirezionata verso il file.

8.11.2 Ridirezione in input

\$ Comando < file: l'input per il comando viene letto dal file specificato.

Implementazione Anche in questo caso, la *shell* crea un processo figlio che compie i seguenti passaggi:

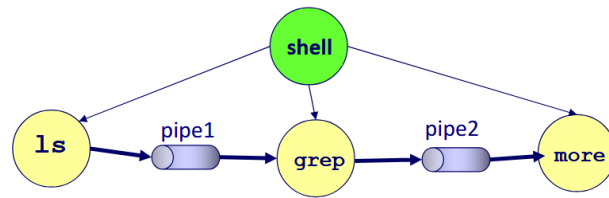
1. Chiude il *file descriptor* 0 (associato allo `stdin`).
2. Apre il file di input, che assumerà il *file descriptor* 0.
3. Esegue il comando tramite la *system call* `exec`.

In questo modo, ogni lettura da `stdin` effettuata dal comando verrà reindirizzata al contenuto del file specificato.

8.11.3 Piping di comandi

Comando₁ | Comando₂ | ... | Comando_n: il risultato di Comando_{n-1} viene utilizzato come input da Comando_n.

Implementazione La *shell* crea un processo figlio per ogni comando da eseguire. Tra ogni processo ed il precedente, poi, crea un canale di comunicazione *pipe* in maniera tale che lo *stdout* del *Comando_n* sia lo *stdin* del *Comando_{n+1}*.



8.11.4 Dup

Nome	dup
Descrizione	Duplica un elemento della tabella dei file
Parametri	fd – <i>file descriptor</i> del file da duplicare
Valore di ritorno	fd – nuovo <i>file descriptor</i> -1 – fallimento
Libreria	unistd.h
Firma	<pre>#include <unistd.h> int dup(int fd);</pre>

Conseguenze Quando viene eseguita, la *system call* *dup* duplica una voce nella tabella dei file aperti del processo, creando un nuovo *file descriptor* che fa riferimento alla stessa entry del file originale. Il nuovo descrittore è il primo disponibile, assegnato in ordine crescente rispetto ai già esistenti.

9 Gestione della Memoria

All'interno di sistemi multiprogrammati, è necessario che il SO gestisca la memoria in modo da consentire la presenza contemporanea di più processi, tenendo in considerazione i seguenti aspetti:

- Velocità di accesso
- Grado di multiprogrammazione
- Grado di utilizzo della memoria
- Protezione delle aree di memoria assegnate ai processi e al *kernel*

9.1 Compiti del Sistema Operativo

I compiti legati alla gestione della memoria centrale, affidati al sistema operativo, sono i seguenti:

- **Allocazione** della memoria ai processi
- **Deallocazione** della memoria
- Separazione e **protezione** degli spazi di indirizzamento associati ai processi
- Realizzazione del **binding** tra indirizzi logici e fisici
- Gestione di spazi di indirizzamento logico più ampi rispetto alla memoria fisica disponibile (**memoria virtuale**)

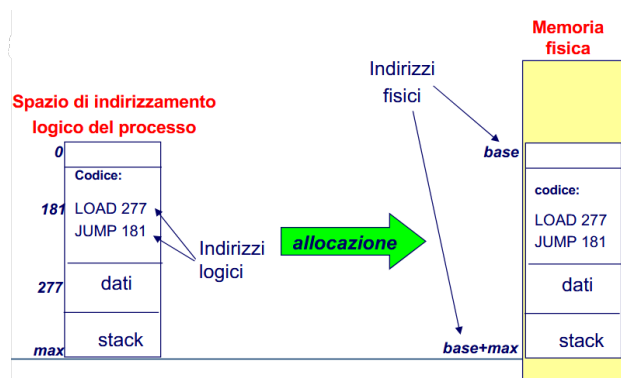
Dal punto di vista del dispositivo fisico, le operazioni logiche associate alla memoria centrale sono essenzialmente **load** e **store** di dati e istruzioni all'interno di un vettore di celle.

9.2 Spazio di Indirizzamento

A ciascun processo viene associato uno spazio di indirizzamento logico, necessario per contenere codice, dati e stack. Questo spazio è distinto da quello fisico e consente una maggiore flessibilità nella gestione della memoria da parte del sistema operativo.

9.2.1 Esecuzione

Definito lo spazio di indirizzamento, si definisce l'**esecuzione** di un processo come l'allocazione in memoria fisica dello spazio di indirizzamento logico.



9.2.2 Indirizzi logici e fisici

Si definisce la seguente differenza tra tipo di indirizzi:

- **Logico**: riferimento nello spazio logico di indirizzamento del processo
- **Fisico**: riferimento assoluto alla cella in memoria (a livello *hardware*)

All'interno della forma sorgente dei programmi, si definiscono anche gli **indirizzi simbolici**. Questi associano aree di memoria a nomi simbolici (nomi variabili, funzioni, ecc.) e vengono trasformati in indirizzi logici durante la fase di compilazione.

9.2.3 Binding

Con il termine **binding** si intende il processo di associazione tra indirizzi logici e indirizzi fisici. Esistono due approcci principali:

- **Binding statico:** gli indirizzi fisici vengono determinati in fase di compilazione. Alternativamente, il compilatore genera indirizzi logici che, durante il caricamento, vengono convertiti in indirizzi assoluti dal *loader*.
- **Binding dinamico:** l'associazione tra indirizzi logici e fisici viene effettuata durante l'esecuzione. In questo contesto, lo spazio di indirizzamento di un processo può essere rilocato dinamicamente nella memoria centrale.

Caricamento dinamico L'obiettivo principale del **binding dinamico** è ottimizzare l'utilizzo della memoria. In particolare, è possibile caricare funzioni o procedure in *runtime*, ovvero solo nel momento in cui vengono effettivamente invocate, mediante l'utilizzo di un *loader* di collegamento.

Il sistema operativo si occupa di:

- Gestire i problemi di visibilità e accesso relativi alla condivisione di funzioni
- Garantire la protezione tra gli spazi di memoria appartenenti a processi diversi
- Supportare, ove necessario, la condivisione controllata di aree di memoria tra più processi

9.2.4 Tecniche di allocazione su memoria centrale

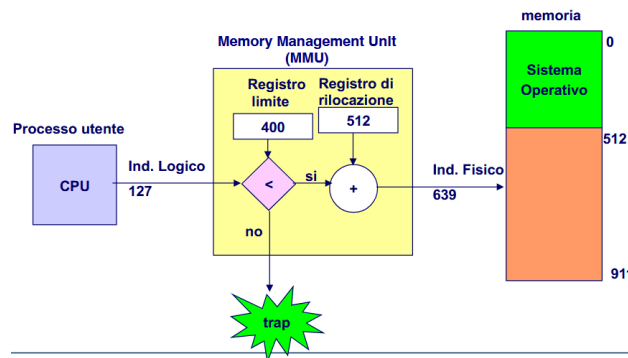
Esistono differenti tecniche di allocazione sulla memoria centrale:

- **Contigua:** a partizione singola o multipla.
- **Non contigua:** paginazione e segmentazione.

9.3 Allocazione contigua a partizione singola

La memoria disponibile per l'allocazione dei processi utente non è partizionata: un solo processo alla volta può essere caricato in memoria.

Solitamente il SO risiede nella parte di memoria bassa ($[0, N_{max}]$) e necessita protezione dagli accessi dei processi utente. A tal fine viene utilizzato il **registro di rilocazione**, contenente il primo indirizzo disponibile per il processo ($N_{max} + 1$).

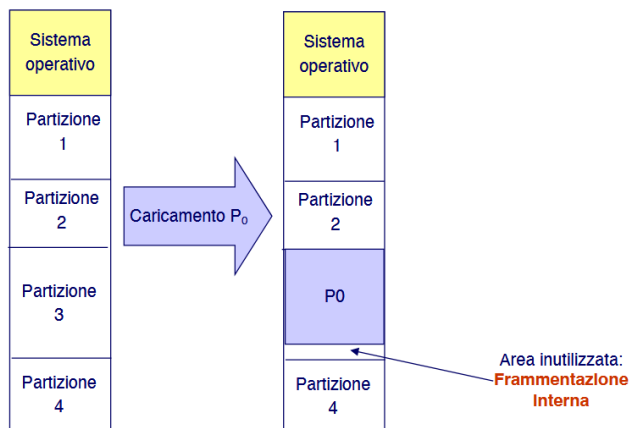


9.4 Allocazione contigua a partizione multipla

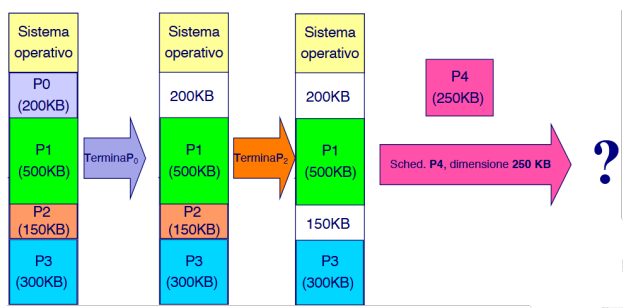
Ad ogni processo viene associata un'area di memoria distinta, detta **partizione**. Quest'ultima può essere:

- **Fissa:** la memoria fisica viene a priori divisa in n (fissato) partizioni. Il numero di ogni partizione è fissato ed è compito del SO cercare la partizione adatta ad accogliere un processo.
- **Variabile:** ogni partizione è creata e dimensionata sulla base del processo da allocare. Quando viene caricato un processo, il SO verifica la presenza di uno spazio di memoria sufficientemente grande per inserire lo stesso.

Problemi partizione fissa La partizione fissa causa frammentazione interna ed un grado di multiprogrammazione limitato. La dimensione massima dello spazio di indirizzamento di un processo, poi, è limitata alla dimensione della partizione stessa.



Problemi partizione variabile Le partizioni variabili causano frammentazione esterna, necessità di compattazione periodica e lo sviluppo di una politica di allocazione.



9.4.1 Protezione

La protezione della memoria viene realizzata a livello *hardware* tramite due registri speciali: il registro di rilocalizzazione (*RR*) e il registro di limite (*RL*).

Ad ogni processo viene assegnata una coppia di valori $\langle V_{RR}, V_{RL} \rangle$, dove:

- V_{RR} rappresenta l'indirizzo iniziale della partizione di memoria assegnata al processo
- V_{RL} rappresenta la dimensione della partizione

Quando un processo viene caricato in memoria, il *dispatcher* carica i valori V_{RR} e V_{RL} nei registri *RR* ed *RL*. Questi valori vengono poi utilizzati per verificare che ogni accesso alla memoria da parte del processo rientri nei limiti consentiti. Qualsiasi tentativo di accedere a un indirizzo al di fuori di tali limiti genera un'interruzione (*trap*) gestita dal sistema operativo.

9.5 Allocazione non contigua con paginazione

La *paginazione* consiste nel dividere lo spazio fisico in blocchi di dimensione fissa detti **frame**. A questi vengono associate porzioni di processo, anch'esse suddivise in blocchi di uguale dimensione detti **pagine**. Si distinguono quindi due tipi di spazio:

- **Spazio fisico:** suddiviso in *frame* di dimensione D_f
- **Spazio logico:** suddiviso in *pagine* di dimensione D_f

Ogni pagina logica viene mappata su un frame fisico tramite una tabella delle pagine, che tiene traccia dell'associazione tra pagina logica e frame fisico.

9.5.1 Vantaggi

La paginazione elimina la frammentazione esterna, poiché le pagine logiche contigue possono essere mappate su *frame* fisicamente non contigui. Inoltre, riduce la frammentazione interna, grazie alla dimensione fissa dei *frame*. Un ulteriore vantaggio consiste nella possibilità di caricare solo un sottoinsieme delle pagine logiche di un processo in memoria.

9.5.2 Supporto hardware

indirizzo logico Un indirizzo logico è composto da:

- **p**: numero della pagina logica $(m - n)$ bit
- **d**: offset rispetto all'inizio della pagina n bit

Questi valori determinano:

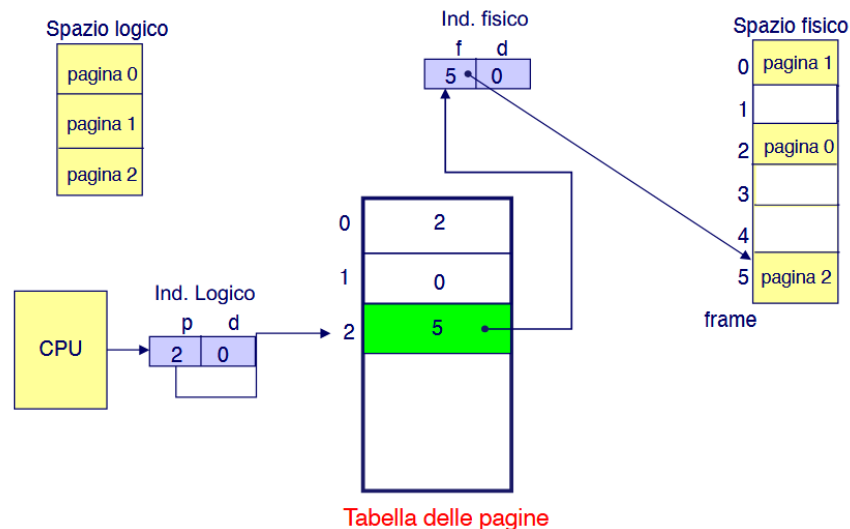
- Dimensione massima dello **spazio logico** di indirizzamento: 2^m
- Dimensione della **pagina**: 2^n
- **Numero** totale di pagine: 2^{m-n}

indirizzo fisico Un indirizzo fisico è anch'esso diviso in due componenti:

- **f**: numero di *frame* (pagina fisica)
- **d**: offset rispetto all'inizio del frame

9.5.3 Tabella delle pagine

Il collegamento tra indirizzi logici e fisici viene effettuato tramite una **tabella delle pagine**, che mantiene per ogni pagina logica il corrispondente frame fisico. Ogni processo possiede una propria tabella.

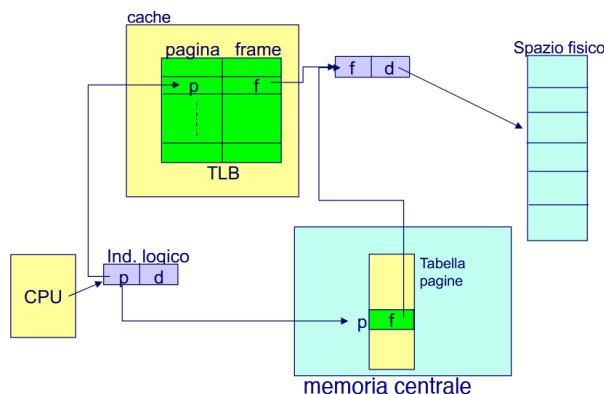


Poiché la tabella può essere di grandi dimensioni, per rendere efficiente la traduzione degli indirizzi si utilizzano tre strategie principali:

- **Registri CPU**: nei casi in cui la tabella sia di dimensioni molto contenute.
- **Memoria centrale**: il registro *PTBR* (Page Table Base Register) memorizza l'indirizzo base della tabella in memoria centrale. Ogni accesso alla memoria richiede due letture: una alla tabella e una ai dati.
- **Memoria centrale e cache**: utilizzo dei *TLB* (Translation Look-aside Buffers), cache associative ad alta velocità che memorizzano traduzioni recenti.

9.5.4 Translation Look-aside Buffers (TLB)

La tabella di paginazione è in memoria centrale e una copia degli elementi più acceduti è inserita all'interno della cache (TLB).



Gestione La *TLB* viene riempita gradualmente. Inizialmente vuota, aumenta di dati con l'esecuzione del processo e l'accesso a nuove pagine.

Si definisce *hit-ratio* la percentuale media di volte in cui una pagina viene trovata all'interno della *TLB*.

9.5.5 Protezione

Avendo la tabella delle pagine lunghezza fissa, è necessario poter distinguere tra *entry* valide ed *entry* inutilizzate. Al tal fine, vengono utilizzati un *bit* di validità (con valore pari a uno per *entry* significative, zero altrimenti) e un *Page Table Length Register* (con valore pari al numero di *entry* valide).

Esistono ulteriori bit di protezione sull'accesso alle pagine, ad esempio la possibilità che una pagina sia *read-only*.

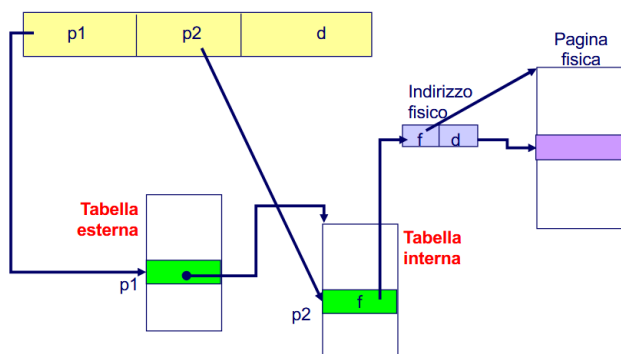
9.5.6 Paginazione a più livelli

La paginazione a più livelli consente un'allocatione non contigua anche della tabella delle pagine, riducendo lo spreco di memoria. In questa struttura, la tabella delle pagine è suddivisa in più livelli: la tabella di primo livello (detta esterna) contiene puntatori a tabelle di secondo livello (interne), che a loro volta contengono gli indirizzi dei *frame* fisici.

Un esempio tipico, considerando indirizzi logici a 32 bit e pagine di 4 KiB (2^{12} byte), prevede la seguente suddivisione dell'indirizzo logico:

- p1: indice nella tabella esterna (10 bit)
- p2: indice nella tabella interna (10 bit)
- d: offset all'interno della pagina (12 bit)

Tale struttura permette 2^{10} pagine di primo livello e 2^{10} tabelle di secondo livello (ognuna con 2^{10} *entry*).



Vantaggi Questa tecnica permette di indirizzare spazi logici di dimensioni elevate e aggiunge la possibilità di mantenere in memoria esclusivamente le tabelle interne.

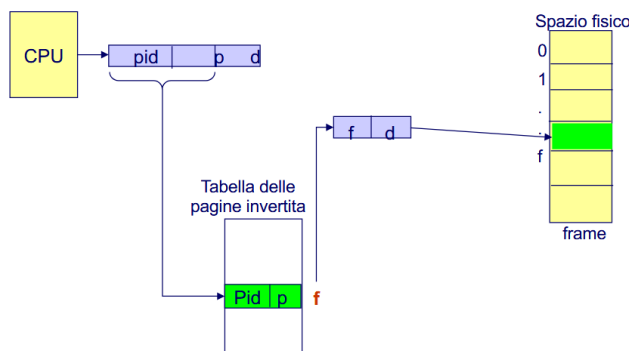
Svantaggi La paginazione a più livelli richiede un tempo d'accesso più alto.

9.5.7 Tabella delle pagine invertita

Per limitare l'uso della memoria necessaria per mantenere le tabelle delle pagine, alcuni SO utilizzano la tabella delle pagine invertita. In questo approccio, ogni voce della tabella rappresenta un frame fisico (il cui numero coincide con la posizione della voce nella tabella). Se il frame è occupato, la voce conterrà le seguenti informazioni:

- **pid**: identificatore del processo a cui è assegnato il frame
- **p**: numero della pagina logica del processo

Per tradurre un indirizzo logico $\langle \text{pid}, p, d \rangle$, si ricerca nella tabella la voce che contiene la coppia $\langle \text{pid}, p \rangle$. L'indice della voce trovata rappresenta il numero del frame fisico a cui è mappata la pagina logica p . L'offset d viene poi aggiunto per ottenere l'indirizzo fisico finale.



Problemi Tale tecnica aggiunge un ulteriore tempo di ricerca all'interno della tabella e difficoltà di condivisione dei frame tra processi.

9.6 Segmentazione

La **segmentazione** è una tecnica di gestione della memoria che suddivide lo spazio di indirizzamento di un processo in **segmenti** logici, basati sul significato semantico dei dati (ad esempio: codice, dati, *stack*, *heap*, ecc.). Ogni segmento può essere allocato in memoria in modo contiguo, può avere attributi differenti (ad esempio: segmento codice $\rightarrow$ *read-only*) ed è identificato da un numero intero gestito dal SO.

9.6.1 Struttura

In un sistema segmentato, ogni indirizzo logico ha la forma $\langle \text{segmento}, \text{offset} \rangle$, dove:

- **segmento**: identificatore numerico del segmento a cui si riferisce l'indirizzo
- **offset**: posizione relativa della cella all'interno del segmento

9.6.2 Supporto hardware

A livello hardware, il sistema utilizza una **tabella dei segmenti** per ciascun processo. Ogni entry della tabella contiene le seguenti informazioni:

- **Base**: indirizzo di inizio del segmento in memoria fisica
- **Limite**: dimensione del segmento (limite massimo dell'**offset**)

Il sistema utilizza il registro *STBR* (*Segment Table Base Register*) per memorizzare l'indirizzo di partenza della tabella dei segmenti del processo attualmente in esecuzione.

Realizzazione Per evitare una singola tabella globale eccessivamente grande, il sistema operativo mantiene una tabella per ogni processo. L'accesso alla tabella dei segmenti può essere ottimizzato tramite:

- **Registri della CPU**
- **Segment Table Base Register (STBR)** e **Segment Table Length Register (STLR)**
- **Memoria centrale + cache** per velocizzare le operazioni di traduzione

9.6.3 Segmentazione paginata

La segmentazione così presentata è un'estensione della tecnica di allocazione a partizioni variabili, passando da una singola partizione per processo a più partizioni (segmenti) per processo. Questo però comporta il presentarsi degli stessi problemi legati al partizionamento originale, come la frammentazione esterna. Per risolvere tali problematiche, segmentazione e paginazione possono essere combinate:

- Lo **spazio logico** è suddiviso in **segmenti**
- Ogni segmento è ulteriormente diviso in **pagine** (dimensione costante)

Vantaggi Eliminazione della frammentazione esterna e non necessità di tenere in memoria l'intero segmento

9.6.4 Segmentazione in Linux

I sistemi Linux sfruttano una segmentazione paginata a tre livelli, con vari tipi di segmento:

- **Code:** per *user* e *kernel*
- **Data:** per *user* e *kernel*
- **Task state segments:** registri dei processi per il cambio di contesto
- etc.

9.7 Overlay

La tecnica dell'*overlay* consente di mantenere contemporaneamente in memoria solo le istruzioni e i dati necessari alla fase corrente di esecuzione, privilegiando quelli utilizzati più frequentemente.

In alcuni casi, infatti, la memoria disponibile potrebbe non essere sufficiente per contenere l'intero programma. In tal caso, il programmatore suddivide manualmente il programma in più parti (**overlay**), che vengono caricate e scaricate dinamicamente durante l'esecuzione dal **gestore di overlay**.

Questa tecnica consente l'esecuzione di programmi più grandi della memoria fisica disponibile.

9.8 Memoria virtuale

La **memoria virtuale** è una tecnica di gestione della memoria che permette l'esecuzione di processi non completamente allocati in memoria, in maniera trasparente rispetto al programmatore e ai processi stessi.

9.8.1 Vantaggi

Una tecnica di questo tipo presenta molteplici vantaggi:

- **Dimensione** dello spazio logico **svincolata** dalla grandezza della memoria
- Gradi di **multiprogrammazione indipendente** dalla memoria
- **Efficienza:** caricamento e *swap* hanno un costo ridotto
- **Astrazione**

9.9 Paginazione su richiesta

Con la **paginazione su richiesta**, tutte le pagine di un processo risiedono inizialmente nella memoria secondaria (*backing store*). Alcune di esse vengono successivamente trasferite in memoria centrale, all'occorrenza, dal *pager*.

9.9.1 Pager

Il *pager* è la componente del Sistema Operativo incaricata del trasferimento delle pagine tra memoria secondaria e memoria centrale (funge da *swapper* di pagine). Nel caso della paginazione su richiesta, il *pager* è definito *lazy*, poiché carica in memoria centrale solo le pagine effettivamente necessarie all'esecuzione.

9.9.2 Funzionamento

Il funzionamento della paginazione su richiesta si articola in due fasi principali:

- **Prima del caricamento** (*swap-in*) di un processo: il *pager* può tentare di prevedere quali pagine saranno necessarie, caricandole anticipatamente.
- **Durante l'esecuzione** di un processo: il *pager* gestisce il caricamento delle singole pagine su richiesta, mentre lo *swapper* si occupa dell'intero processo in caso di necessità di memoria.

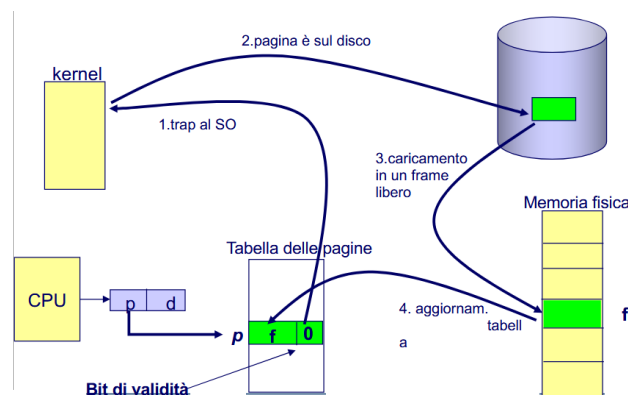
Hardware necessario La paginazione su richiesta richiede la presenza di memoria secondaria, strutture per la sua gestione e le tabelle delle pagine (*PTBR*, *PTLR*, *TLB*, etc.).

9.9.3 Page Fault

All'interno della tabella delle pagine viene inserito un bit di validità per distinguere le pagine presenti in memoria centrale da quelle residenti in memoria secondaria. Quando, durante la traduzione di un indirizzo logico, la pagina richiesta non risulta caricata in memoria centrale, viene generata un'interruzione detta **page fault**.

Quando il SO riceve un *page fault*, esegue tipicamente le seguenti operazioni:

1. **Salvataggio** del contesto del processo in esecuzione
2. **Verifica** della causa del *page fault*
 - (a) **Riferimento illegale**: la pagina richiesta viola le politiche di protezione → generazione di errore
 - (b) **Riferimento legale**: la pagina è presente in memoria secondaria
3. **Caricamento** della pagina in un frame libero della memoria centrale
4. **Aggiornamento** della tabella delle pagine (bit di validità e frame associato)
5. **Ripristino** del contesto e ripresa dell'esecuzione del processo dal punto in cui era stato interrotto



9.9.4 Sovrallocazione

Se successivamente ad un *page fault* non dovessero esserci più *frame* liberi per la nuova pagina, allora il sistema si dice in **sovrallocazione**. Per risolvere tale situazione, il SO:

1. **Individua** la pagina vittima $P_{vittima}$
2. **Salva** $P_{vittima}$ sul disco
3. **Carica** di P_{nuova} nel frame liberato
4. **Aggiorna** le tabelle
5. **Riprende** il processo

9.9.5 Dirty Bit

Per evitare il salvataggio in memoria secondaria di pagine che non sono state modificate (come pagine di codice o dati non alterati), viene introdotto un ulteriore bit di controllo nella tabella delle pagine: il **dirty bit**. Questo indica se la pagina è stata modificata (*dirty*) dopo il caricamento in memoria e, durante una fase di *swap-out*, ne viene controllato il valore dal SO per decidere se scrivere o meno la pagina nella memoria secondaria.

Algoritmi di sostituzione Esistono diversi algoritmi per la scelta della pagina vittima, tra cui:

- **LFU** (*Least Frequently Used*): viene sostituita la pagina usata meno frequentemente in un determinato intervallo temporale. Ogni pagina ha associato un contatore degli accessi, e la pagina con il valore più basso viene selezionata come vittima.
- **FIFO** (*First In, First Out*): viene rimpiazzata la pagina residente in memoria da più tempo. È necessario mantenere una struttura (come una coda) che memorizzi l'ordine di caricamento delle pagine.
- **LRU** (*Least Recently Used*): viene selezionata la pagina che non è stata utilizzata da più tempo. Richiede un meccanismo per tracciare l'ordine degli accessi alle pagine. Può introdurre un certo *overhead*, in quanto ogni accesso a una pagina comporta l'aggiornamento delle informazioni temporali.

9.9.6 Implementazione algoritmo LRU

L'algoritmo *LRU* può essere implementato in due principali modalità:

- **Time Stamping**: per ogni pagina viene memorizzato un *timestamp* relativo all'ultimo accesso. La pagina con il valore di *timestamp* più vecchio viene selezionata come vittima. Questo metodo riduce il costo della selezione, ma introduce un *overhead* per l'aggiornamento del tempo a ogni accesso.
- **Stack**: le pagine vengono mantenute in una struttura a *stack*, dove la pagina più recentemente usata si trova in cima. Ogni accesso sposta la pagina in cima allo stack. In questo modo, la pagina meno recentemente utilizzata è sempre in fondo allo *stack*. Questo approccio elimina il costo della ricerca della vittima, ma comporta un elevato *overhead* di gestione dello *stack*, proporzionale al numero di frame.

Linux Alcuni SO, come Linux, adottano una versione approssimata dell'algoritmo *LRU*. A ciascuna pagina è associato un *bit di riferimento*, impostato a 1 ogni volta che la pagina viene acceduta e periodicamente azzerato da un meccanismo di scansione. In fase di sostituzione, le pagine con il bit di riferimento a 0 sono candidate preferenziali per l'eliminazione. In alcuni casi, viene inoltre considerato anche il *dirty bit*.

9.9.7 Thrashing

La paginazione su domanda carica una pagina solamente se strettamente necessaria. Questo può comportare **thrashing**, ovvero il SO impiega più tempo per la paginazione che per l'esecuzione.

9.10 Working Set

Per ridurre il fenomeno del *thrashing*, sono state introdotte tecniche di **pre-paginazione**, come la previsione delle pagine che un processo utilizzerà nella prossima fase di esecuzione. Una di queste tecniche è il concetto di *working set*.

Si osserva che un processo, in un dato intervallo temporale, tende a:

- Utilizzare un sottoinsieme relativamente piccolo del proprio spazio di indirizzamento logico.
- Modificare gradualmente, nel tempo, l'insieme delle pagine utilizzate.

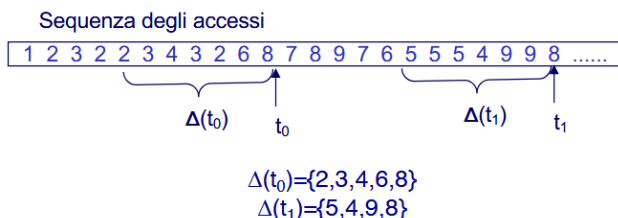
Da queste osservazioni derivano i due principi di **località**:

- **Località spaziale**: alta probabilità di accedere a locazioni di memoria vicine a quelle recentemente accedute.
- **Località temporale**: alta probabilità di riaccedere a locazioni di memoria utilizzate di recente (ad esempio, in cicli o iterazioni).

9.10.1 Funzionamento

Il concetto di *working set* si basa sul principio di località temporale. Dato un intero Δ , il *working set* di un processo P all'istante t è definito come l'insieme delle pagine **indirizzate** da P negli ultimi Δ riferimenti alla memoria:

$$WSP(t, \Delta) = \{\text{pagine referenziate da } P \text{ tra } t - \Delta \text{ e } t\}$$



All'avvio, il SO carica in memoria un *working set* iniziale per ogni processo. Durante l'esecuzione, tale insieme viene aggiornato dinamicamente, tenendo traccia delle ultime Δ pagine effettivamente utilizzate. L'obiettivo è mantenere in memoria solo le pagine realmente necessarie al processo, riducendo così il numero di *page fault*.

9.10.2 Valore di Δ

Il parametro Δ discrimina l'efficienza del sistema:

- Δ **piccolo**: *working set* non garantisce località (alto numero di *page fault*).
- Δ **grande**: *working set* sovrabbondante.

Definita la dimensione di Δ , per ogni processo P_i si individua un WSS_i (*working set*) e si definisce:

$$\text{richiesta totale di frame: } D = \sum_i WSS_i$$

Dato m numero totale di *frame* liberi:

- $D < m$: c'è spazio per allocare nuovi processi
- $D > m$: *swap* di uno o più processi necessario

9.11 UNIX

9.11.1 Versioni iniziali

Nelle prime versioni del sistema UNIX, la gestione della memoria si basava su un'allocazione **contigua** dei segmenti, priva di memoria virtuale ma con supporto per la **condivisione del codice**.

In presenza di difficoltà nell'allocazione della memoria, il sistema ricorreva allo *swapping* dell'intero processo: veniva temporaneamente rimosso dalla memoria centrale e spostato nella memoria secondaria.

L'algoritmo adottato per l'allocazione della memoria, sia centrale che secondaria, era di tipo **first fit**, ovvero il primo spazio libero sufficientemente grande veniva selezionato per l'allocazione.

9.11.2 Swapping

In assenza di memoria virtuale, uno **swapper** si occupa, periodicamente, della gestione di *swap-in* e *swap-out* dei processi:

- **swap-in**: processi inattivi, "ingombranti" e da più tempo in memoria.
- **swap-out**: processi piccoli e da più tempo *swapped*.

9.11.3 Versioni attuali

Nelle versioni moderne, UNIX utilizza una gestione della memoria basata su **segmentazione paginata**, implementata tramite **memoria virtuale** e **paginazione su richiesta**. I segmenti non sono più allocati in maniera contigua.

Pre-paginazione Il sistema può caricare anticipatamente, nei frame liberi, pagine che si prevede verranno utilizzate a breve (*pre-paging*). In caso di *page fault*, se la pagina risulta già caricata, è sufficiente aggiornare la tabella delle pagine e la lista dei frame liberi, senza dover accedere alla memoria secondaria.

Core map È una struttura dati mantenuta dal *kernel* che descrive lo stato di ciascun frame di memoria centrale. Viene consultata, ad esempio, per individuare frame liberi o assegnati durante la gestione dei *page fault*.

Page fault Grazie al **bit di validità** associato a ciascuna pagina nella tabella, il sistema può determinare l'origine del *page fault*, distinguendo tra:

- **Riferimento illegale**: la pagina richiesta non è valida; il processo viene terminato per violazione di protezione.
- **Pagina non presente**: la pagina è valida ma non caricata; viene eseguito lo *swap-in* dalla memoria secondaria.
- **Pagina pre-caricata**: la pagina è già in memoria; si procede solo all'aggiornamento della tabella delle pagine.

9.11.4 Algoritmo di sostituzione

I sistemi UNIX utilizzano una variante semplificata dell'algoritmo *LRU*, che impiega un **bit di utilizzo** associato a ciascuna pagina. Questo bit è inizialmente impostato a 0 e viene portato a 1 ogni volta che la pagina viene acceduta.

Quando il sistema deve selezionare una pagina vittima, viene scelta una pagina con bit di utilizzo pari a 0. Successivamente, il bit di utilizzo di tutte le altre pagine viene azzerato.

Pagedaemon Il processo incaricato della sostituzione delle pagine è chiamato *pagedaemon*. Quando è necessario liberare memoria, il *pagedaemon* seleziona una pagina vittima, la rende invalida e, se il *dirty bit* è impostato (cioè uguale a 1) o non presente, ne effettua il salvataggio in memoria secondaria prima di procedere alla sua rimozione.

In generale, si definiscono 3 parametri negli algoritmi di sostituzione:

- **lotsfree**: numero di *frame* liberi per evitare la sostituzione di pagine
- **minfree**: numeri minimo di *frame* liberi necessari per evitare lo *swapping*
- **desfree**: numero desiderato di *frame* liberi

$$lotsfree > desfree > minfree$$

9.11.5 Scheduling, paginazione e swapping

Lo *scheduler* può richiedere la sostituzione di pagine quando il numero di *frame* liberi scende al di sotto della soglia **lotsfree**. Se invece il sistema di paginazione risulta sovraccarico, ossia:

- $frame\ liberi < minfree$ oppure
- $frame\ liberi\ nell'unità\ di\ tempo < desfree$

allora viene attivato lo *swapper*, al massimo una volta ogni secondo, per liberare memoria.

Il sistema operativo cerca comunque di limitare l'impatto del *pagedaemon*, evitando che utilizzi più del 10% del tempo della CPU. Per questo motivo, il *pagedaemon* viene attivato, al massimo, ogni 250 ms.

9.12 Linux

Nei sistemi Linux, l'allocazione della memoria è basata su **segmentazione paginata**. Le tabelle delle pagine sono organizzate su **2 o 3 livelli**, mentre i moduli di codice caricati dinamicamente vengono allocati in maniera **contigua**. La memoria virtuale è gestita **senza l'utilizzo del working set**.

9.12.1 Organizzazione della memoria fisica

Linux mantiene alcune aree riservate della memoria fisica, ciascuna con uno scopo specifico:

- **Codice del kernel:** area contenente il codice eseguibile del kernel. Le relative pagine sono bloccate in memoria (*locked*) e non sono soggette a paginazione.
- **Cache del kernel:** heap del kernel, anch'esso bloccato in memoria.
- **Area dei moduli gestiti dinamicamente:** per il caricamento e la gestione dei moduli del kernel.
- **Buffer cache:** utilizzata per la gestione dell'I/O su dispositivi a blocchi.
- **Inode cache:** contiene copie degli *i-node* utilizzati di recente.
- **Page cache:** contiene pagine in disuso, in attesa di essere sostituite.
- **Altre aree:** riservate a funzionalità specifiche del kernel e dei driver.

La restante memoria è interamente destinata ai processi utente.

9.12.2 Spazio di indirizzamento

In un sistema Linux, lo spazio di indirizzamento massimo assegnabile a un processo è di **4 GiB**, suddiviso come segue:

- **3 GiB** riservati allo **spazio di indirizzamento virtuale utente**
- **1 GiB** riservato al **kernel**, accessibile solo in modalità *kernel mode*

Lo spazio di indirizzamento del processo è organizzato in sezioni contigue e omogenee. Ogni sezione è costituita da un insieme di pagine che condividono le stesse proprietà in termini di protezione e politiche di paginazione.

Ogni pagina ha una dimensione fissa, generalmente pari a **4 KiB** nelle architetture Intel.

9.12.3 Paginazione

La paginazione è strutturata **su tre livelli**. Progettata per supportare i processori Alpha, su alcune piattaforme può essere ridotta a due livelli.

9.12.4 Page-fetching e sostituzione

Linux **non adotta la tecnica del *working set*** ma mantiene un **insieme di pagine libere**, noto come *page cache*.

Analogamente a quanto avviene nei sistemi UNIX, viene controllato una volta al secondo che il numero di pagine libere sia sufficiente per garantire le prestazioni del sistema.

9.13 MS Windows XP

Nel sistema Windows XP, la gestione della memoria virtuale sfrutta una tecnica detta **page clustering**: in caso di *page fault*, non viene caricata solo la pagina richiesta, ma anche un insieme di pagine adiacenti (*page cluster*).

A ogni processo è associato un **working set**, definito da un numero minimo e massimo di pagine. Quando il numero di frame liberi scende al di sotto di una certa soglia critica, il sistema operativo attiva un meccanismo di recupero detto **working set trimming**. Questo consiste nella riduzione del *working set* dei processi, in particolare di quelli che superano il minimo consentito.

10 File Comandi in UNIX

10.1 Shell

La *shell* è un programma che funge da interfaccia tra l'utente e il SO, consentendo l'interazione tramite l'inserimento di comandi. Può essere vista sia come un interprete di comandi provenienti dallo **standard input** (o da file) sia come un ambiente di esecuzione per un linguaggio di scripting potente e flessibile.

10.1.1 Molteplici shell

Ogni SO può mettere a disposizione differenti *shell*. Ogni utente può, poi, scegliere la preferita attraverso il comando **chsh**, che memorizza all'interno del file `/etc/passwd` la preferenza.

La *shell* default di Linux è **bash** (`/bin/bash`).

10.1.2 Shell di login

La *shell* di login è responsabile della gestione dell'accesso al sistema da parte dell'utente. Dopo aver ricevuto e verificato le credenziali fornite, il sistema avvia un processo associato alla *shell* preferita dell'utente, che verrà eseguita con i privilegi e l'ambiente di quest'ultimo. Al termine della procedura di login, l'utente viene posizionato nel proprio direttorio iniziale, tipicamente specificato nel file `/etc/passwd`.

10.1.3 Comandi

I principali comandi a disposizione dell'utente si trovano tipicamente all'interno della directory `/bin`. Tuttavia, è possibile estendere le funzionalità del sistema attraverso la creazione di script personalizzati. In ambiente UNIX, i comandi si suddividono generalmente in due categorie:

- **Built-in**: comandi integrati nella *shell*, eseguiti direttamente come funzioni.
- **External**: comandi esterni alla *shell*, presenti nel file system e avviati tramite la creazione di un processo figlio, eseguito tramite la **exec**.

10.1.4 Logout

Tipicamente, per chiudere una *shell* basta invocare il comando **exit**. Per uscire dalla *shell* di login, invece, è necessario effettuare il logout oppure inviare il segnale di EOF (CTRL + D) oppure SIGINT (CTRL + C).

10.1.5 Comandi

I principali comandi a disposizione degli utenti si trovano all'interno della cartella `/bin`. È possibile anche svilupparne ulteriori attraverso lo scripting. Generalmente, comunque, si definiscono due tipi di comandi:

- **built-in**: definiti internamente ed eseguiti come funzione
- **external**: eseguiti dalla *shell* attraverso la creazione di un figlio dedicato all'esecuzione del comando

10.2 Comandi shell Linux

I comandi UNIX si comportano come filtri, ovvero producono un risultato su uno o più output dato un solo input. Alcuni dei comandi più usati sono:

- **grep** <testo> [<file>...]: ricerca di testo
- **tee** [<file>...]: scrive l'input sia su video che su file
- **awk** '<pattern> action' [<file>...]: ricerca testo ed esegue comandi
- **sort** [<file>...]: ordina alfabeticamente
- **rev** [<file>...]: inverte l'ordine delle righe
- **cut** [-options] [<file>...]: seleziona colonne
- **head** [-options] [<file>...]: filtra la "testa" del file
- **tail** [-options] [<file>...]: filtra la "coda" del file
- **expr** <expression>: valuta un'espressione

10.2.1 Piping

Con il *piping*, l'output di un comando può essere reindirizzato come input di un altro comando. Generalmente, può essere utilizzato su *shell* attraverso | (pipe), ma la sua implementazione può variare:

- **DOS**: vengono realizzati file temporanei utilizzati in maniera ordinata dai file
- **UNIX**: viene utilizzato il costruito *pipe*

10.2.2 Espressioni

È possibile indicare alla *shell* di valutare il risultato di un comando e sostituirlo con il comando stesso attraverso l'utilizzo di ` (backtick). Ad esempio, `echo risultato = `expr 5 + 1`` stamperà sullo standard output: `risultato = 6`.

10.3 Metacaratteri

La *shell* riconosce caratteri speciali detti *wild card*:

- *****: stringa di zero o più caratteri in un nome di file
- **?**: un carattere nel nome di un file
- **[zfc]**: un qualunque carattere, in un nome di file, compreso nell'insieme (es. [a-z] lettera compresa tra 'a' e 'z')
- **#**: commento (fino alla fine della linea)
- **** (Escape): il carattere successivo non è da considerare come speciale (**inibizione**)

10.4 Variabili

All'interno delle *shell* è anche possibile definire variabili, sempre e solo contenenti stringhe. È possibile riferirsi a una variabile indicando il suo nome preceduto da \$:

- **VAR**: nome della variabile (*l-value*)
- **\$VAR**: valore della variabile (*r-value*)

Per assegnare un valore a una variabile è necessario indicarne il nome e il valore, separati da uguale. Per esempio, `x=italia` assegna alla variabile `x` il valore stringa `'italia'`.

10.4.1 Ambiente

Con ambiente si intende l'insieme di **variabili d'ambiente** definite. Ogni comando vive nell'ambiente associato alla *shell* che lo sta eseguendo, potendo così fare riferimento alle variabili definite (che esse siano standard o *user-defined*). Ogni *shell* eredita l'ambiente dalla *shell* che l'ha creata.

È possibile visualizzare tutte le variabili d'ambiente (e i loro valori) attraverso il comando **set**.

10.5 Espansione

Al fine di interpretare correttamente le *wild card* e le espressioni, le *shell* intraprendono un ulteriore passaggio prima dell'esecuzione di un comando, detto **espansione**. Attraverso questo processo possono essere sostituiti caratteri speciali ed espressioni con il loro relativo valore.

10.5.1 Sequenza

Per eseguire una corretta espansione, le *shell* seguono il seguente procedimento:

1. Sostituzione comandi tra ` ` (backquotes)
2. Sostituzione delle variabili con il loro valore
3. Sostituzione dei metacaratteri con i nomi dei file

10.6 Shell scripting

In ambiente UNIX è possibile sviluppare file di comandi, detti *shell script*, contenenti una sequenza di istruzioni da eseguire all'interno di una *shell*. Ogni riga dello script rappresenta un comando o una struttura di controllo, interpretata sequenzialmente. Gli *shell script* supportano controllo di flusso (condizionali e cicli), l'utilizzo di variabili e il passaggio di argomenti. Il comportamento e la sintassi dello script dipendono dalla *shell* utilizzata per l'esecuzione.

10.6.1 Esecuzione

La *shell* crea un processo figlio e gli assegna l'esecuzione di ogni comando presente all'interno dello *script*. È possibile anche scegliere quale *shell* utilizzare per l'esecuzione attraverso la prima riga del file, che deve seguire la seguente sintassi: `#!<path alla shell>` (i caratteri `#!` vengono interpretati dal SO come direttive dell'interprete).

10.6.2 Argomenti

Gli argomenti passati per l'esecuzione di uno *script* sono accessibili come normali variabili inserendo `$0`, `$1`, `$2`, etc. La variabile `$0`, in particolare, contiene il nome del processo. Generalmente, vengono forniti una serie di comandi per elaborarne il contenuto.

shift Attraverso il comando `shift` ogni variabile d'argomento (`$0`, `$1`, `$2`, etc) assume il valore della sua successiva, tranne `$0`.

set Con l'istruzione `set` si può indicare un insieme di variabili da cui assegnare il valore agli argomenti in ingresso (a partire da `$1`):

```
set var1 var2 var3 → $1=var1, $2=var2, $3=var3.
```

10.6.3 Variabili notevoli

Oltre agli argomenti, vengono messi a disposizione dalla *shell* una serie di ulteriori variabili notevoli:

- `$*`: tutte le variabili posizionali a partire da `$1`
- `$#`: numero di argomenti passati in ingresso (escluso `$0`)
- `$$`: pid del processo in esecuzione
- `$?`: valore intero restituito dall'ultimo comando

10.6.4 Input e output

È possibile effettuare letture (stringhe) da input e assegnare i valori letti a delle variabili:

```
read var1 var2 var3
```

È possibile anche restituire dei dati in standard output:

```
echo il valore della variabile e': $var1
```

10.7 Test

Viene fornito dalle *shell* un comando per la valutazione delle espressioni: `test`. Questo restituisce `true` per i valori pari a 0, altrimenti `false`.

```
test A = A → $? vale 0
test A = B → $? vale 1
```

10.7.1 Parametri più comuni

- `test -f <nomefile>`: esiste il file
- `test -d <nomefile>`: esiste il direttorio
- `test -r <nomefile>`: diritto di lettura sul file
- `test -z <stringa>`: stringa nulla
- `test <stringa>`: stringa non nulla
- `test <stringa1> = <string2>`
- `test <val1> -gt <val2>`: maggiore
- `test <val1> -lt <val2>`: minore
- `test <val1> -le <val2>`: minore uguale
- `test <val1> -ge <val2>`: maggiore uguale

10.7.2 Alternativa

Al posto del comando `test` è possibile utilizzare le parentesi quadre ed inserire al loro interno gli stessi argomenti del comando originale:

```
test -d mydir → [ -d mydir ]
```

È altresì possibile utilizzare le doppie parentesi quadre (*keyword* del linguaggio e non comando) per sostituire `test` ed implementare:

- operatori `&&` e `||`
- operatori `>` e `<` (solo per le stringhe)
- metacaratteri
- *regular expressions* attraverso l'operatore `=~`

10.8 Strutture di controllo

10.8.1 If

```
if <condizioni>
then
    <comandi>
[elif <condizioni>
then
    <comandi>]
[else
    <comandi>]
fi
```

oppure

```
if <condizioni>; then
    <comandi>
[elif <condizioni>; then
    <comandi>]
[else
    <comandi>]
fi
```

10.8.2 Alternativa multipla

Nelle alternative multiple viene offerta la possibilità di fare *pattern matching*.

```
case <var> in
    <pattern-1>
        <comandi>;;
    <pattern-i> | <pattern-j> | <pattern-k>
```

```
    <comandi>;;  
    <pattern-n>  
    <comandi>;;  
esac
```

10.8.3 Cicli enumerativi

Con i cicli enumerativi è possibile ciclare sulla lista di argomenti in input oppure su una lista di stringhe.

```
for <var> [in <list>]  
do  
    <comandi>  
done
```

10.8.4 Cicli non enumerativi

Viene anche offerta la possibilità di eseguire cicli basati su condizione: **while** e **until**. Questi seguono la stessa sintassi ma con logica invertita: **while** continua finché la condizione è vera, **until** il contrario.

```
while <condizione o list-comandi>  
do  
    <comandi>  
done
```

```
until <condizione o list-comandi>  
do  
    <comandi>  
done
```

11 Programmazione Concorrente in modello ad ambiente globale

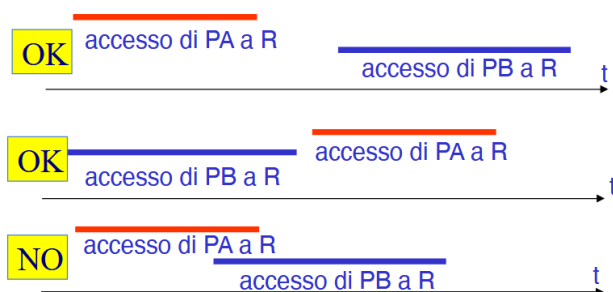
Nel modello ad ambiente globale, il termine risorsa si identifica con quello di **variabile** allocata nella memoria comune. In un modello a memoria comune:

- Ogni processo è identificato dal termine *thread*
- La comunicazione avviene attraverso dati e risorse condivisi

È compito del linguaggio di programmazione, in un ambiente di questo tipo, fornire strumenti e costrutti (costruiti su meccanismi realizzati dal SO) per gestire i problemi legati alla sincronizzazione della concorrenza.

11.1 Mutua esclusione

Avere risorse condivise tra più *thread* pone un problema relativo all'accesso esclusivo contemporaneo su quella risorsa. Con la **mutua esclusione** si impone che esclusivamente un processo possa accedere a una risorsa condivisa in un determinato periodo di tempo.



11.2 Operazioni indivisibili

Un'operazione è detta **indivisibile** (o **atomica**) se opera su una risorsa comune a più *thread* e che non può essere acceduta contemporaneamente da nessun altro *thread*.

11.3 Sezione critica

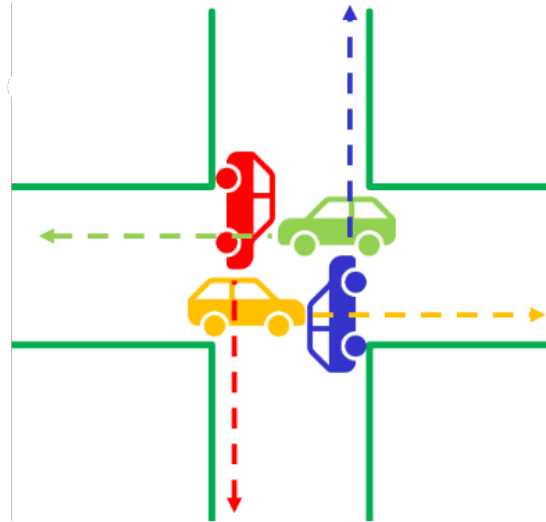
L'insieme di istruzioni con cui un *thread* modifica un insieme di risorse comuni è detta **sezione critica**. Ad un insieme di variabili comuni possono essere associate una sola sezione critica o più sezioni critiche (classe di sezioni critiche).

Sezioni critiche appartenenti alla stessa classe devono escludersi mutuamente nel tempo. Ovvero, una sola sezione critica di una classe può essere in esecuzione, ad ogni istante.

11.4 Deadlock

Si definisce **deadlock** la situazione nella quale, su un insieme di processi, ogni processo è in attesa di un evento che può essere causato solo da un altro processo dell'insieme.

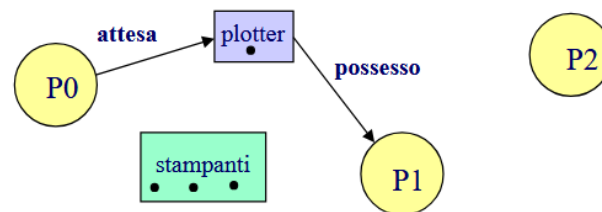
Si supponga una situazione nella quale tutte le risorse sono occupate da processi che, per procedere con la loro esecuzione, richiedono accesso ad ulteriori risorse (a quel punto già occupate da altri processi in attesa).



11.4.1 Grafi di allocazione delle risorse

Al fine di evitare *deadlock*, è possibile sviluppare grafi di allocazione delle risorse ai processi:

- **Cerchi:** Processi
- **Rettangoli:** Risorse
- **Archi:**
 - **Risorsa → Processo:** possesso della risorsa (allocazione)
 - **Processo → Risorsa:** attesa della risorsa (richiesta)



11.4.2 Condizioni necessarie

Per potersi presentare un blocco critico è necessario che quattro differenti condizioni, necessarie e sufficienti, si verifichino:

1. **Mutua esclusione:** le risorse sono utilizzate in maniera mutuamente esclusiva
2. **Possesso e attesa:** ogni processo che possiede una risorsa ne può chiedere un'altra
3. **Impossibilità di prelazione:** una volta assegnata ad un processo, una risorsa non può essere sottratta (no *preemption*)
4. **Attesa circolare:** esiste un gruppo di processi $P_0, P_1, \dots, P_N$ in cui P_0 attende una risorsa posseduta da P_1 , che attende una risorsa posseduta da P_2 , che attende ...

11.4.3 Trattamento

Esistono due approcci per il trattamento di blocchi critici:

- **Prevenzione:** si evita a priori il verificarsi di *deadlock*. Può essere di tipo **statico** o **dinamico**.
- **Rilevazione e ripristino:** il SO rileva il *deadlock* e applica algoritmi di risoluzione

Prevenzione statica Con la prevenzione statica si impone che almeno una delle quattro condizioni necessarie per il *deadlock* non si verifichi. In particolare, si evita l'attesa circolare implementando un criterio di allocazione gerarchica delle risorse: un processo non può acquisire una risorsa R_j se è già in possesso di una risorsa R_k , con $k > j$.

Prevenzione dinamica Il SO previene i *deadlock* attraverso specifici algoritmi, come quello del banchiere: una sequenza di processi $\{P_0, P_1, \dots, P_n\}$ è salva se, per ogni processo P_i , le richieste di risorse che P_i può ancora effettuare possono essere soddisfatte con le risorse attualmente libere più le risorse allocate a tutti i processi P_j con $j < i$.

Rilevazione e ripristino A differenza della prevenzione, la rilevazione e il ripristino dei *deadlock* richiedono dal SO la capacità di individuare blocchi in corso e attuare soluzioni a posteriori, come: terminazione forzata dei processi (*rollback*) oppure prelazione delle risorse allocate nel *deadlock*. Soluzioni di questo tipo sono generalmente più costose.

11.5 Soluzioni al problema della mutua esclusione

Al fine di produrre una corretta soluzione al problema della mutua esclusione è necessario che essa:

- Esegua in maniera mutuamente esclusiva sezione critiche appartenenti alla stessa classe
- Impedisca ai processi all'esterno di sezioni critiche di bloccare l'accesso
- Non presenti *deadlock*

11.5.1 Struttura generale

La struttura generale di una soluzione deve essere scomponibile in tre distinti punti:

- **Prologo:** il processo P verifica la disponibilità della risorsa R . Se questa è libera allora la occupa, altrimenti attende la sua liberazione
- **Sezione critica.**
- **Epilogo:** il processo P dichiara la risorsa R libera.

11.5.2 Possibili soluzioni

È possibile implementare diverse soluzioni che permettono di gestire la mutua esclusione:

- **Algoritmiche:** la soluzione non necessita particolari strutture o meccanismi di sincronizzazione ma solo della condivisione delle variabili.
- **Hardware-base:** il supporto è fornito dall'architettura.
- **Basate su strumenti di sincronizzazione:** prologo ed epilogo vengono gestiti attraverso strutture di sincronizzazione offerte dal *kernel* del SO, le quali permettono la sospensione dei processi in attesa di eseguire sezioni critiche.

11.6 Soluzioni algoritmiche

Le soluzioni algoritmiche si basano sull'assunzione che l'*hardware* garantisca la mutua esclusione su operazioni di lettura e scrittura di *word*.

11.6.1 Algoritmo di Dekker

L'algoritmo di Dekker è stata la prima soluzione al problema della mutua esclusione. Si supponga che i due processi condividano:

```
int busy1 = 0, busy2 = 0;
int turno = 1; /* dominio {1,2} */
```

```
/* Processo 1 */
```

```
main() {
    ...
    busy1 = 1;
    while (busy2 == 1) {
        if (turno == 2) {
            busy1 = 0;
            while (turno != 1);
            busy1 = 1;
        }
    }
    <sezione critica>
    turno = 2;
    busy1 = 0;
    ...
}
```

```
/* Processo 2 */
```

```
main() {
    ...
    busy2 = 1;
    while (busy1 == 1) {
        if (turno == 1) {
            busy2 = 0;
            while (turno != 2);
            busy2 = 1;
        }
    }
    <sezione critica>
    turno = 1;
    busy2 = 0;
    ...
}
```

11.6.2 Algoritmo di Peterson

Si supponga che i processi condividano:

```
int busy1 = 0, busy2 = 0;
int turno = 1; /* dominio {1,2} */
```

```
/* Processo 1 */
```

```
main() {
    ...
    busy1 = 1;
    turno = 2;
    while (busy2 && turno == 2);
    <sezione critica>
    busy1 = 0;
    ...
}
```

```
/* Processo 2 */
```

```
main() {
    ...
    busy2 = 1;
    turno = 1;
    while (busy1 && turno == 1);
    <sezione critica>
    busy2 = 0;
    ...
}
```

11.7 Attesa attiva

Le soluzioni algoritmiche realizzano l'attesa con dei cicli. Di fatto, ogni processo utilizza inutilmente la CPU anche quando aspetta: **attesa attiva** (*busy waiting*);

11.7.1 Alteriori requisiti

Viste le soluzioni algoritmiche, vengono aggiunti due nuovi requisiti per lo sviluppo di una soluzione alla mutua esclusione:

- Devono essere **eliminate** tutte le forme di **attesa attiva**, sospendendo l'esecuzione di un processo in attesa d'accesso alla sezione critica.
- Ad un processo che vuole accedere ad una sezione critica è **garantito l'accesso** in un tempo finito (no *starvation*).

11.8 Soluzione hardware

11.8.1 Disabilitazione interrupt

Tra le soluzioni *hardware*, è presente la **disattivazione degli interrupt** durante le sezioni critiche. In particolare, si attuerebbe la disabilitazione nel prologo e la riattivazione nell'epilogo.

Problemi Una soluzione con disabilitazione degli interrupt sarebbe valida esclusivamente per processi che operano sullo stesso processore, eliminerebbe ogni possibilità di concorrenza e renderebbe il sistema insensibile agli stimoli esterni durante le sezioni critiche.

11.8.2 Test and Set

Molti processori presentano all'interno della loro *ISA* un'istruzione che permette di leggere e modificare il valore di un registro con una sola istruzione: **test_and_set**. Questa è indivisibile, della durata di un ciclo di memoria e possibile soluzione della mutua esclusione.

L'esecuzione segue la seguente struttura:

```
int test_and_set(int *shared_mem) {
    int result = *shared_mem;
    *shared_mem = 0;
    return result;
}
```

Attraverso l'utilizzo di questa istruzione è possibile sviluppare meccanismi di **lock** e **unlock**.

11.8.3 Lock e Unlock

Sia x una variabile associata ad una classe di sezioni critiche, inizialmente libera:

- $x = 0 \rightarrow$ risorsa occupata
- $x = 1 \rightarrow$ risorsa libera

Si definiscono allora le seguenti operazioni **atomiche**:

```
void lock(int *x) {
    while(!*x);
    *x=0;
}
```

```
void unlock(int *x) {
    *x=1;
}
```

Con questa struttura, però, il lock non garantirebbe un corretto funzionamento della mutua esclusione. Si presenta, allora, la versione con **test_and_set**:

```
void lock(int *x) {
    while(!test_and_set(x));
}
```

Si supponga a questo punto che due processi condividano una variabile `int x = 1`:

```
/* Processo 1 */
```

```
main() {
    ...
    lock(&x);
    <sezione critica>
    unlock(&x);
    ...
}
```

```
/* Processo 2 */
```

```
main() {
    ...
    lock(&x);
    <sezione critica>
    unlock(&x);
    ...
}
```

Concorrenzialità Quando due processi eseguono contemporaneamente una **lock**, è l'hardware ad occuparsi della sequenzialità di esecuzione. Questa soluzione è quindi applicabile in ambienti a multiprocessore.

Problemi Una soluzione con **lock** e **unlock** presenta problemi di **attesa attiva** e non un'implicita risoluzione alla **starvation**.

11.9 Soluzioni software

La mutua esclusione è anche raggiungibile attraverso strumenti software offerti dal *kernel* SO, come:

- Semafori
- Monitor

11.10 Semafori

Un semaforo è uno strumento, generalmente offerto dal *kernel* del SO, che permette di risolvere qualunque problema di sincronizzazione tra processi. In particolare, con semaforo si intende un dato astratto s , al quale è possibile accedere esclusivamente attraverso due operazioni atomiche:

- $p(s)$: ritarda il processo finché s vale 0, poi ne decrementa il valore di 1
- $v(s)$: incrementa di 1 in valore di s

Essendo le operazioni atomiche, il valore del semaforo può essere modificato esclusivamente da un processo alla volta.

11.10.1 Implementazione

È importante che un semaforo soddisfi due principali requisiti:

1. Eliminazione dell'**attesa attiva**: i processi che non possono accedere alla sezione critica deve essere sospeso ed inserito in una coda associata al semaforo
2. Eliminazione della **starvation**: politica *FIFO* sulla scelta del processo da risvegliare

Essendo il gestore dei processi, il *kernel* può soddisfare entrambi i requisiti.

È possibile a questo punto immaginare un semaforo secondo la seguente struttura:

```
typedef struct {
    unsigned int value;
    queue processQueue;
} Semaphore;
```

Su questo opereranno le seguenti operazioni:

```
void p(Semaphore *s) {
    if (s->value == 0) {
        <inserisci il processo corrente in s->processQueue>;
        <sospendi il processo corrente>;
    } else {
        s->value--;
    }
}
```

```
void v(Semaphore *s) {
    if (<coda s->processQueue non vuota>) {
        <rimuovi il primo processo da s->processQueue>;
        <risveglia il processo rimosso>;
    } else {
        s->value++;
    }
}
```

11.10.2 Atomicità p e v

Durante l'esecuzione di un'operazione offerta da un semaforo da un processo è importante che nessun altro processo possa accedere alle stesse operazioni. È necessario, cioè, che p e v siano realizzate come operazioni indivisibili attraverso l'uso di lock e unlock:

```
typedef struct {
    unsigned int value;
    queue processQueue;
    int lock; // inizializzato a 1 e detto "semaforo di mutua esclusione"
} Semaphore;
```

```
void p(Semaphore *s) {
    lock(s->lock);
    if (s->value == 0) {
        <inserisci il processo corrente in s->processQueue>;
        <sospendi il processo corrente>;
    } else {
        s->value--;
    }
    unlock(s->lock);
}
```

```
void v(Semaphore *s) {
    lock(s->lock);
    if (<coda s->processQueue non vuota>) {
        <rimuovi il primo processo da s->processQueue>;
        <risveglia il processo rimosso>;
    } else {
        s->value++;
    }
    unlock(s->lock);
}
```

11.10.3 Casi d'uso

Oltre al problema della mutua esclusione, i semafori possono essere utilizzati per qualunque tipo di problema riguardi la sincronizzazione tra processi. Ad esempio, la cooperazione tra processi concorrenti può essere risolta mediante l'uso di semafori.

11.10.4 Problemi

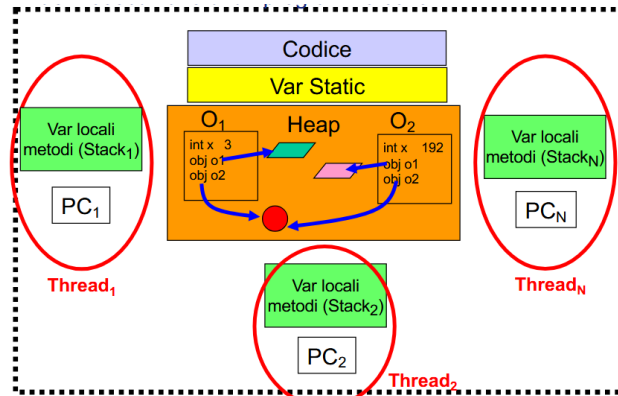
L'utilizzo di semafori potrebbe talvolta risultare eccessivamente "di basso livello". Il numero di semafori, poi, tende a crescere insieme alla complessità della politica di sincronizzazione, il che favorisce la possibilità di errori.

Vengono per questo motivo offerti ulteriori costrutti linguistici per la sincronizzazione:

- **monitor**
- **regioni critiche condizionali**
- etc.

12 Java thread (1)

I *thread* in *Java* sono processi attivi, ognuno dei quali con la propria sequenza di istruzioni. Ogni programma è composto da almeno un *thread*, corrispondente all'esecuzione del metodo `main`, ed è possibile crearne dinamicamente ulteriori.



12.1 Implementazione

Il linguaggio *Java* offre due modalità per implementare i *thread*:

- Estensione della classe `Thread`
- Implementazione dell'interfaccia `Runnable`

12.1.1 Thread

I *thread* sono oggetti offerti dalla classe `java.lang.Thread` e posseggono un metodo `run()`, il quale definisce il codice che il *thread* dovrà eseguire.

Per creare un nuovo *thread* è necessario istanziare un nuovo oggetto della classe definita e richiamarne il metodo `start()`

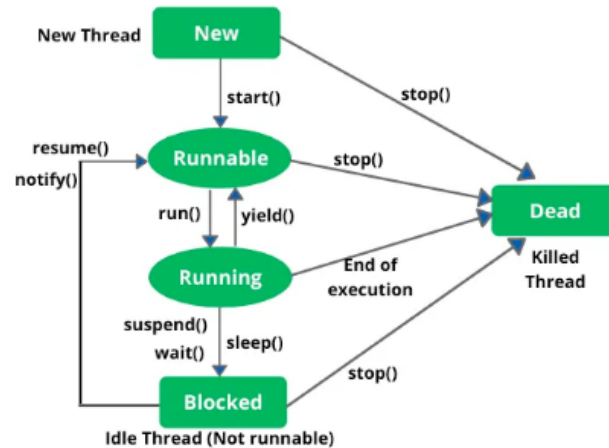
```
class CustomThread extends Thread {
    public void run() {
        <istruzioni da eseguire>
    }
}

public class Main {
    public static void main(String[] args) {
        CustomThread thread = new CustomThread();
        thread.start();
        <resto del programma>
    }
}
```

Con il codice mostrato vengono creati 2 *thread* concorrenti: il principale (associato al `main`) e un'istanza di `CustomThread`.

Ciclo di vita di un thread Un *thread* in java segue un ciclo di vita che presenta i seguenti stadi:

- **New**: creazione dell'oggetto `Thread`
- **Runnable**: *thread* pronto per l'esecuzione
- **Running**: esecuzione in corso
- **Blocked**: in attesa di un evento
- **Dead**: morte naturale del *thread* o invocazione di `stop()`



12.1.2 Runnable

Un ulteriore metodo per creare *thread* in *Java* consiste nell'implementazione dell'interfaccia `Runnable`. Questa permette di non dover obbligatoriamente avere una classe *thread* che eredita da `java.lang.Thread`, in maniera tale da essere più flessibile.

```

class CustomThread implements Runnable {
    public void run() {
        <istruzioni da eseguire>
    }
}

public class Main {
    public static void main(String[] args) {
        Thread thread = new Thread(new CustomThread());
        thread.start();
        <resto del programma>
    }
}

```

In questo caso, per eseguire il *thread* è necessario utilizzare un oggetto di tipo `java.lang.Thread` con argomento del costruttore il *thread* desiderato. Successivamente, come per le normali classi `java.lang.Thread`, è sufficiente richiamare il metodo `start()`.

12.2 Sincronizzazione di thread

Trovandosi in un modello ad ambiente globale, i *thread* devono sincronizzarsi utilizzando oggetti comuni. Le possibili interazioni sono:

- **Competitivo** (mutua esclusione): *object locks* e metodi *synchronized*
- **Cooperativo**: meccanismi *wait*→*notify* e variabili condizione

12.2.1 Join

Un *thread* può sincronizzarsi con la terminazione di un altro *thread* attraverso l'invocazione, su quest'ultimo, del metodo `join()`.

12.2.2 Synchronized

Per ogni oggetto *Java*, la *JVM* associa automaticamente un *object lock*, utilizzato per indicarne lo stato di utilizzo. È possibile definire come critici alcuni blocchi di codice che operano su tale oggetto mediante la

keyword **synchronized**. Attraverso questo meccanismo, il compilatore inserisce automaticamente un prologo e un epilogo per gestire il blocco e lo sblocco dell'oggetto.

```
Object mutexLock = new Object();
```

```
public void function() {  
    <codice non critico>  
    synchronized (mutexLock) {  
        <sezione critica>  
    }  
    <codice non critico>  
}
```

```
Object mutexLock = new Object();  
  
synchronized public void function() {  
    <sezione critica>  
}
```

Viene associato un **lock** ad ogni oggetto sincronizzato. Ogni processo che prova ad accedere alla sezione critica, quando quest'ultima è già occupata, viene quindi sospeso ed inserito nell'*entry set* associata all'oggetto, in attesa della liberazione.

Quando un processo esce dalla sezione critica viene eseguito automaticamente un **unlock**, se non sono presenti *thread* all'interno della *entry set*. Se così non dovesse essere, viene svegliato un *thread* senza disattivare il lock.

12.2.3 Semafori

Java presenta la propria implementazione dei semafori attraverso la classe `java.util.concurrent.Semaphore`. Alle operazioni primitive **p** e **v** sono associati i metodi **acquire** e **release**.

```
Semaphore semaphore = new Semaphore(n);
```

```
public function method() {  
    semaphore.acquire();    // p()  
    <sezione critica>  
    semaphore.release();    // v()  
}
```

12.3 Stop

Anche se deprecato, *Java* implementa il metodo **stop()** per terminare l'esecuzione dei *thread*. L'utilizzo è tuttavia sconsigliato vista la possibilità di stati inconsistenti nel caso in cui la terminazione avvenga all'interno di sezioni critiche.

12.4 Suspended

Così come **stop()**, *Java* offre anche il metodo **suspend()** per bloccare l'esecuzione dei processi, almeno fino all'esecuzione di una **resume()**. Anche tale metodo è deprecato poiché, in caso di sospensione, le risorse utilizzate non vengono liberate.

12.5 Ulteriori metodi dei thread

- **sleep(long ms)**: sospende il *thread* per un numero definito di millisecondi
- **interrupt()**: invia un evento al *thread* che ne causa l'interruzione
- **interrupted()/isInterrupted()**: verificano se il *thread* è stato interrotto
- **isAlive()**: informa sullo stato di vita del *thread*
- **yield()**: costringe il *thread* a cedere il controllo della CPU

13 Monitor

Il *monitor* è un costrutto che associa un insieme di operazioni (dette *entry*) ad una struttura dati comune a più processi, tale che:

- Le operazioni *entry* siano le uniche permesse sulla struttura
- Le operazioni *entry* siano mutuamente esclusive

Un *monitor* è dotato, poi, di variabili locali, permanenti rispetto alle *entry*, e operazioni *non-entry* (entrambe utilizzabili solamente all'interno del monitor).

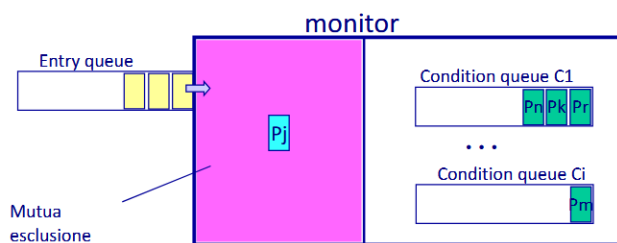
13.1 Utilità

I *monitor* rappresentano un meccanismo utilizzato per controllare l'accesso concorrente a risorse condivise da parte di più processi concorrenti. L'accesso avviene secondo politiche predefinite poiché le risorse non sono più accessibili direttamente dai processi, ma esclusivamente tramite le operazioni sicure fornite dal monitor stesso.

13.2 Sincronizzazione

La sincronizzazione in un *monitor* viene sviluppata in due livelli:

1. Garanzia di un **unico accesso contemporaneo** al *monitor*. Il costrutto è responsabile dell'interruzione dei processi che fanno accesso in caso di risorsa occupata, inserendoli nella *entry queue*.
2. **Ordine di accesso** alla risorsa secondo una condizione di sincronizzazione. Quando tale condizione non è soddisfatta, il processo deve essere sospeso nella *condition queue*.



13.2.1 Implementazione

Il primo livello di sincronizzazione viene sviluppato dal linguaggio attraverso il concetto di *entry* (intrinsecamente mutuamente esclusive). Il secondo livello, invece, è onere del programmatore, al quale viene predisposto uno strumento di sincronizzazione: variabili condizione (*condition*).

13.3 Variabili condizione

Una variabile condizione (*condition*) rappresenta una coda di sospensione per i processi che non soddisfano i requisiti di sincronizzazione. È possibile operare su una *condition* esclusivamente attraverso due operazioni prestabilite: *wait(cond)* e *signal(cond)*.

13.3.1 wait

Il processo viene sospeso e inserito nella coda associata alla *condition*. Il monitor viene liberato.

13.3.2 signal

Un processo presente nella coda associata alla *condition* viene risvegliato. Se non sono presenti processi in coda, la funzione non produce effetti.

13.4 Semantiche dell'operazione `signal`

Quando un processo esegue un'operazione `signal`, potrebbe verificarsi una temporanea violazione del principio di mutua esclusione, poiché sia il processo che effettua la `signal` sia quello risvegliato dalla `wait` potrebbero risultare attivi all'interno del *monitor*. Per risolvere questa ambiguità, sono state proposte due semantiche alternative:

- **`signal_and_wait`**: il processo che effettua la `signal` cede immediatamente il controllo al processo risvegliato, sospendendo la propria esecuzione e inserendosi nella *entry queue* del monitor.
- **`signal_and_continue`**: il processo che esegue la `signal` continua la propria esecuzione, mentre il processo risvegliato viene posto in attesa nella *entry queue* fino a quando non potrà entrare nel monitor. Questo approccio viene utilizzato in **java**

In entrambi i casi, il processo inserito all'interno della *entry queue* non possiede particolare priorità d'esecuzione rispetto agli altri processi della coda

13.4.1 `signal_and_urgent_wait`

Una variante della `signal_and_wait` è rappresentata dalla strategia `signal_and_urgent_wait`. In questo caso, il processo che esegue la `signal` non viene inserito nella normale *entry queue*, bensì in una *urgent queue*, dotata di priorità rispetto alla coda di ingresso standard. Una volta completata l'esecuzione del processo risvegliato, infatti, il controllo torna al processo che ha effettuato la `signal` senza rilasciare la mutua esclusione del *monitor*.

13.4.2 `signal_and_return`

Un'ulteriore variante della `signal_and_wait` è la `signal_and_return`. Quest'ultima viene utilizzata quando il processo risvegliante ha completato la sua esecuzione con la `signal` e quindi cede la priorità al processo risvegliato, senza rilasciare la mutua esclusione del monitor.

13.4.3 `signal_all`

Esiste anche una variante per la funzione `signal_and_continue`, la quale permette di inserire tutti i processi nella *condition queue* all'interno della *entry queue*

14 Java thread (2)

14.1 Variabili condizione

Java mette anche a disposizione l'utilizzo di variabili condizione, ottenibili attraverso l'interfaccia `java.util.concurrent.locks`. Questa offre variabili condizioni dotate di tre metodi principali, equivalenti alle operazioni genericamente associate alle variabili condizione:

```
public interface Condition {
    public void await() throws InterruptedException;
    public void signal();
    public void signalAll();
}
```

La semantica seguita è quella del `signal_and_continue`.

14.1.1 Lock

Sempre attraverso `java.util.concurrent.locks`, viene offerta la possibilità di sviluppare esplicitamente il concetto di *lock*:

```
public interface Lock {
    public void lock();
    public void unlock();
    public void newCondition();
}
```

14.1.2 Utilizzo

A ogni variabile condizione è associato un oggetto *lock*. Questo viene rilasciato ogni volta che un *thread* esegue una chiamata ad `await()`, e viene automaticamente riacquisito quando il *thread* viene risvegliato. Data la stretta correlazione tra *lock* e variabili condizione, la creazione di nuove variabili condizione è delegata agli oggetti *lock*, tramite il metodo `newCondition()`.

14.2 Monitor

Il linguaggio *Java* non fornisce nativamente un costrutto esplicito per i *monitor*, ma ne consente l'implementazione attraverso gli strumenti di sincronizzazione disponibili. La gestione dei dati condivisi può essere realizzata mediante oggetti `Condition`, l'accesso controllato tramite metodi pubblici (le cosiddette *entry*), mentre la mutua esclusione è garantita da un *lock* condiviso.

14.3 Problema dei filosofi a cena

Un classico problema della programmazione *multithreading*, risolvibile con monitor, è quello dei "filosofi" a cena.

14.3.1 Definizione

Cinque filosofi sono seduti attorno a un tavolo circolare. Davanti a ciascuno di essi vi è un piatto di spaghetti e, tra ogni coppia di filosofi adiacenti, si trova una bacchetta. Ogni filosofo alterna due attività: **pensare** e **mangiare**.

Per poter mangiare, un filosofo ha bisogno di entrambe le bacchette alla propria destra e sinistra. Poiché le bacchette sono condivise tra i vicini, è necessario predisporre un meccanismo di sincronizzazione che eviti situazioni di *deadlock* (ovvero blocco reciproco, in cui nessun filosofo riesce a mangiare) e di *starvation* (ovvero il rischio che uno o più filosofi non riescano mai ad accedere alle risorse necessarie per mangiare).

14.3.2 Soluzione problematica

Una possibile soluzione semplice al problema dei filosofi a cena prevede che ciascun filosofo segua il seguente schema:

1. Prende la forchetta alla propria sinistra
2. Prende la forchetta alla propria destra
3. Mangia
4. Ripone entrambe le forchette

Tuttavia, tale soluzione presenta una criticità significativa: il rischio di *deadlock*. Infatti, se tutti i filosofi dovessero contemporaneamente afferrare la forchetta alla loro sinistra, nessuno di essi potrebbe proseguire con la presa della forchetta destra, rimanendo in attesa indefinita di una risorsa che non sarà mai liberata.

14.3.3 Soluzione corretta

Una strategia corretta, invece, prevede che ogni filosofo verifichi la disponibilità di entrambe le forchette prima di acquisirle. Solo nel caso in cui entrambe siano libere, il filosofo procede alla loro presa e quindi alla fase di mangiata.

Tale approccio previene il verificarsi di situazioni di *deadlock*, garantendo che almeno uno dei filosofi possa sempre avanzare, rompendo eventuali circoli di attesa.

15 Gestione della I/O

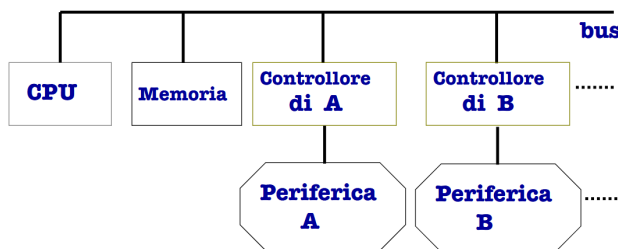
15.1 Sottosistema di I/O

Al sottosistema di I/O in un SO vengono affidati molteplici compiti:

- **Nascondere** al programmatore i dettagli delle interfacce *hardware*
- **Omogeneizzare** la gestione dei dispositivi
- Gestire i **malfunzionamenti**
- Definire lo **spazio dei nomi** (*naming*) con cui vengono identificati i dispositivi
- Garantire la corretta **sincronizzazione** di trasferimento tra i processi e i dispositivi

15.1.1 Nascondere i dettagli

I sottosistemi di I/O tendono a nascondere i dettagli dei dispositivi attraverso vari livelli di astrazione.



15.1.2 Omogenizzazione

Il SO raggruppa i dispositivi in macrogruppi:

- Dispositivi a **carattere** (tastiera, stampante, mouse, etc.)
- Dispositivi a **blocchi** (dischi, nastri, etc.)
- Dispositivi **speciali** (timer, etc.)

15.1.3 Malfunzionamenti

È compito del SO gestire i vari tipi di guasti:

- **Transitori**: disturbi elettromagnetici durante i trasferimenti
- **Permanenti**: rotture
- **Eventi**: mancanza di carta, EOF, etc.

15.1.4 Spazio dei nomi

Lo **spazio dei nomi** rappresenta il meccanismo attraverso cui il SO identifica e gestisce i dispositivi presenti nel sistema. Ogni dispositivo può essere associato a:

- **Nomi simbolici**: etichette assegnate dall'utente o dal sistema per facilitare il riconoscimento del dispositivo (es. `C:`, `/dev/sda`).
- **Identificatori univoci**: valori numerici o stringhe generate automaticamente dal sistema per identificare in modo univoco un dispositivo (es. `UUID`, identificatori hardware).

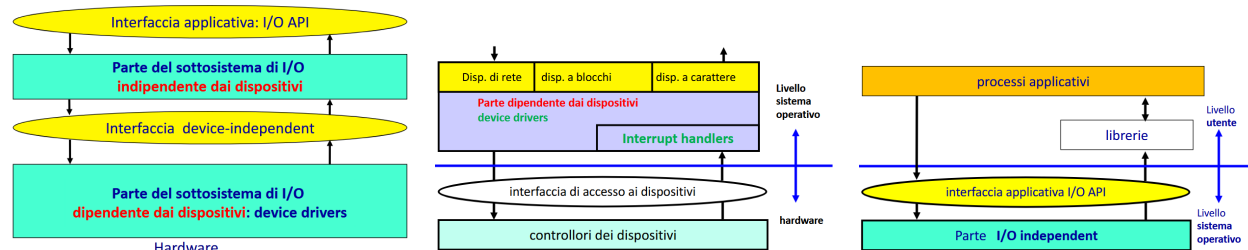
15.1.5 Sincronizzazione

Attraverso la sincronizzazione, il SO deve permettere ai dispositivi e ai processi di comunicare tra loro. La comunicazione può essere:

- **sincrona**: il processo attiva il dispositivo e si blocca in attesa della risposta.
- **asincrona**: il processo attiva il dispositivo e prosegue l'esecuzione senza attendere la risposta.

Tali modalità richiedono la *bufferizzazione* dei dati.

15.2 Architettura

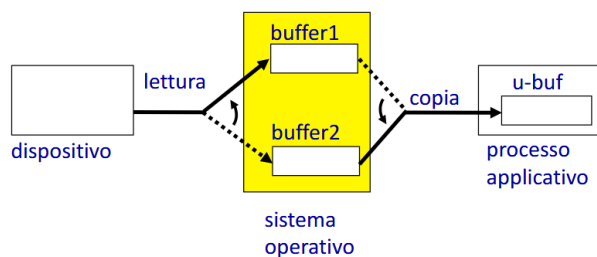


15.3 Buffering

Per ogni trasferimento con I/O, il SO riserva uno spazio di memoria (*buffer*) per contenere temporaneamente i dati coinvolti nel trasferimento. Tale approccio si basa su due esigenze principali:

- Differenza di **velocità** tra processo e periferica, permettendo un *disaccoppiamento* tra le due entità.
- **Quantità** di dati: il processo può richiedere o fornire una quantità di informazioni inferiore rispetto alla capacità del *buffer*.

Lo variabile nello spazio di indirizzamento del processo destinata a contenere i dati trasferiti è detta **u-buf**.



15.4 Gestione malfunzionamenti

Quando un processo tenta di accedere a un dispositivo, possono verificarsi malfunzionamenti dovuti a due tipi di eventi anomali:

- Eventi generati allo **stesso livello** (es. accesso a un dispositivo inesistente)
- Eventi **propagati** dall'*hardware*

15.4.1 Gestione

Per gestire tali eventi, il SO può tentare di **risolvere il problema**, ad esempio mascherando l'evento anomalo, oppure applicare una **gestione parziale** e propagare l'errore all'applicativo, lasciando a questo la responsabilità della gestione finale.

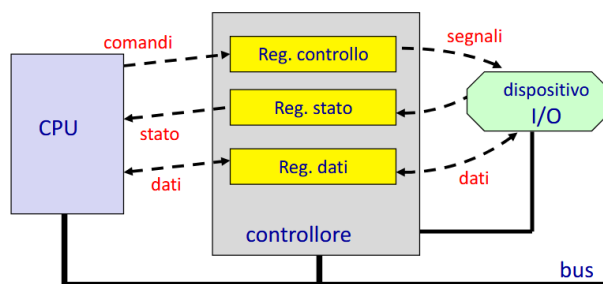
15.5 Allocazione dei dispositivi

I dispositivi si classificano in tre categorie principali:

- **Condivisi** in mutua esclusione.
- **Dedicati** a un solo processo (*server*), al quale altri processi (*client*) possono inviare richieste tramite messaggi.
- In modalità **spooling**: dispositivi virtualizzati tramite gestione in coda dei job (es. stampanti).

15.6 Livello dipendente dai dispositivi

Questo livello offre varie funzioni di interazione con i dispositivi. In particolare, utilizza i gestori dei dispositivi (*device drivers*) per fornire una serie di funzionalità *device-independent*.



15.6.1 Controllore di un dispositivo di I/O

Il controllore è l'elemento hardware che funge da intermediario tra la CPU e il dispositivo di I/O. La sua funzione è gestire i dettagli del funzionamento del dispositivo, ricevere comandi dal SO e restituire informazioni sullo stato o i dati trasferiti. Ogni controllore è associato a una serie di registri specifici:

- registro di **controllo**: utilizzato per controllare la periferica. Immagazzina ogni comando da eseguire e permette l'attivazione tramite il bit di start.
- registro di **stato**: utilizzato per comunicare l'esito di ogni comando. Al termine di ogni comando, viene notificato tramite il bit di flag e può segnalare eventuali errori.
- registro di **dati**: utilizzato per gestire il trasferimento di dati letti o scritti dal dispositivo.

15.7 Modalità di gestione

La sincronizzazione tra CPU e dispositivo può avvenire tramite 2 modelli alternativi:

- Gestione a **controllo di programma** (*pooling*)
- Gestione basata su **interruzioni**

15.7.1 Gestione a controllo di programma

```
processo Esterno {
    while(true) {
        do{} while(bitStart == 0);
        <esegue comando>
        <registra esito ponendo bit di
            fine a 1>
    }
}
```

```
processo Applicativo {
    for(int i=0; i<nDatiDaLeggere; i++)
    {
        <prepara comando>
        <invia comando>
        do{} while(bitFine == 0);
        <verifica esito>
    }
}
```

Problemi I sistemi di gestione a controllo di programma non sono adatti ad ambienti multiprogrammati perchè presentano attesa attiva.

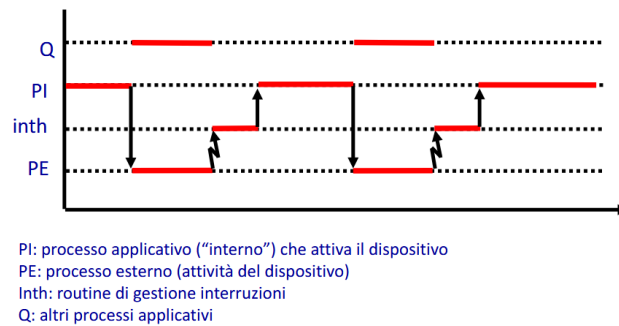
15.7.2 Gestione basata su interruzioni

Per ogni dispositivo viene riservato un semaforo, del quale utilizzare la coda per i processi in attesa di accedere alla periferica. Si attiva poi il dispositivo abilitandolo a interrompere (registro di controllo).

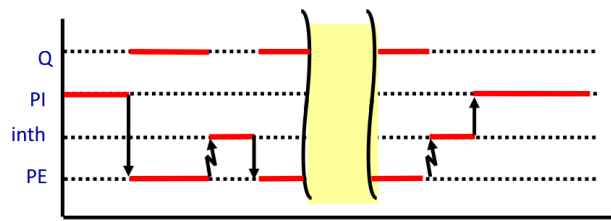
```
processo Interrupt_handler {
    v(dato_disponibile)
}
```

```
processo Applicativo {
    for(int i=0; i<nDatiDaLeggere; i++)
    {
        <prepara comando>
        <invia comando>
        p(dato_disponibile);
        <verifica esito>
    }
}
```

Diagramma temporale La struttura descritta rappresenta un diagramma come segue.



Nei casi ideali, sarebbe opportuno risvegliare il processo applicativo solamente alla fine dell'intero trasferimento dei dati.



15.8 Astrazione di un dispositivo

Un dispositivo può essere astratto come una struttura composta da tre componenti principali:

- **Processi applicativi:** eseguono operazioni di `read` e `write` verso il dispositivo.
- **Device driver:** gestisce gli accessi al dispositivo, elabora gli interrupt e contiene il descrittore associato.
- **Dispositivo:** comunica con l'esterno e segnala il proprio stato tramite interrupt.

15.8.1 Descrittore di un dispositivo

Il descrittore di un dispositivo rappresenta una struttura dati interna al SO utilizzata dal driver per gestire il dispositivo. Contiene i seguenti campi:

- Indirizzo del registro di controllo
- Indirizzo del registro di stato
- Indirizzo del registro dati
- Semaforo (per la disponibilità del dato)
- Contatore dei dati da trasferire
- Puntatore al buffer in memoria
- Esito del trasferimento

15.9 Gestione di dispositivi speciali: il temporizzatore

Per implementare una modalità di esecuzione a divisione di tempo (*time-sharing*), è necessario che il *kernel* gestisca un dispositivo di **temporizzazione** (timer o clock).

15.9.1 Clock

I clock sono dispositivi hardware che generano interruzioni a intervalli regolari, con una frequenza predefinita. La gestione software di tali interruzioni consente al SO di fornire funzionalità avanzate, tra cui:

- Aggiornamento della data e dell'ora di sistema.
- Gestione del quanto di tempo nei sistemi *time-sharing*.
- Monitoraggio dell'utilizzo della CPU da parte dei processi.
- Gestione della `system call ALARM`.
- Gestione dei meccanismi di *time-out* (es. *watchdog timers*).

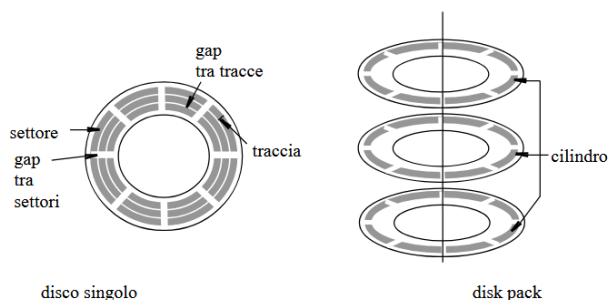
15.9.2 Controllore

Il controllore dei timer, oltre ai normali registri, è dotato di un **registro contatore**. Questo registro viene inizializzato dalla CPU con un valore specifico e decrementato automaticamente dal timer a ogni tick di clock. Quando il valore raggiunge lo zero, il timer genera un'interruzione.

Nel descrittore del dispositivo timer sono presenti anche:

- Un array di N semafori (`fine_attesa[N]`), inizializzati a zero, utilizzati per sospendere i processi che eseguono la chiamata `delay`.
- Un array di N interi (`ritardo[N]`), ciascuno contenente il numero di quanti di tempo rimanenti prima della riattivazione del rispettivo processo bloccato.

15.10 Gestione e organizzazione dei dischi



15.10.1 Indirizzo di un settore

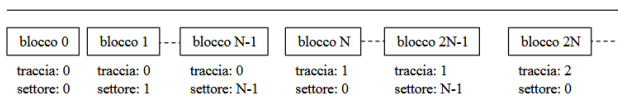
L'indirizzo di un settore segue la forma:

$$(f, t, s)$$

con:

- **f** numero della faccia
- **t** numero della traccia nell'ambito della faccia
- **s** numero del settore all'inizio della faccia

Tale organizzazione permette di ordinare tutti i blocchi di un disco come un array lineare:



15.10.2 Scheduling delle richieste di trasferimento

Per lo *scheduling* delle richieste di trasferimento si definisce:

$$T_f = T_a + T_t$$

dove:

- T_f : tempo totale di trasferimento di un blocco.
- T_a : tempo medio di accesso.
- T_t : tempo necessario per trasferire l'intero blocco sotto la testina.

A sua volta, il tempo di accesso T_a si suddivide in:

$$T_a = S_t + R_l$$

dove:

- S_t : tempo di seek, ovvero il tempo per posizionare la testina sulla traccia corretta.
- R_l : *rotation latency*, ossia il tempo necessario affinché il settore desiderato ruoti sotto la testina.

Il tempo di trasferimento del blocco T_t si definisce come:

$$T_t = \frac{t}{s} \mu s$$

dove:

- t : tempo per un giro completo del disco.
- s : numero di settori per traccia.

Pertanto, la formula finale per il tempo totale di trasferimento T_f diventa:

$$T_f = S_t + R_l + T_t$$

Questo mostra come il tempo di trasferimento dipenda fortemente dal tempo medio di accesso. In particolare, R_l è determinato dalle caratteristiche fisiche del dispositivo, mentre S_t può essere ottimizzato tramite le politiche di gestione delle richieste.

Si individuano due principali strategie di ottimizzazione:

- **Criteri di allocazione dei file:** modalità con cui i blocchi vengono distribuiti sul disco.
- **Politiche di scheduling delle richieste:** ordine con cui le richieste di accesso al disco vengono soddisfatte.

15.10.3 Politiche di scheduling delle richieste

Le richieste di accesso a un dispositivo (tipicamente un disco) possono essere gestite secondo diverse politiche di *scheduling*, ciascuna con vantaggi e svantaggi in termini di tempo medio di servizio e rischio di *starvation*. Le principali sono:

- **FCFS** (*First-Come, First-Served*)
- **SSTF** (*Shortest Seek Time First*)
- **SCAN**
- **C-SCAN** (*Circular SCAN*)

FCFS Le richieste vengono servite nell'ordine in cui arrivano, senza riorganizzazione. Garantisce l'assenza di *starvation*, ma non ottimizza il tempo di accesso e può generare lunghi movimenti inutili della testina.

SSTF Viene selezionata la richiesta più vicina alla posizione attuale della testina, minimizzando il tempo di *seek*. Migliora l'efficienza rispetto a FCFS, ma può causare *starvation* per richieste lontane.

SCAN La testina si muove in una direzione (verso l'estremo del disco), servendo tutte le richieste lungo il percorso, e una volta raggiunto l'estremo inverte la direzione. Riduce il tempo medio di seek rispetto a FCFS e SSTF.

C-SCAN Simile a SCAN, ma una volta raggiunto l'estremo, la testina ritorna direttamente all'inizio del disco (senza servire richieste durante il ritorno) e riprende da lì. Fornisce una maggiore equità, migliorando il tempo di attesa medio rispetto a SCAN per richieste distribuite uniformemente.